Appunti di C

Table of contents

1 Learning Stack — C **8**

I Fondamenti **9**

2 Introduzione al linguaggio C **10**
2.1 Cos'è un programma . . . 10
2.2 Il ciclo di vita di un programma C . . . 10
2.3 Il primo programma . . . 11
2.4 Come compilare ed eseguire . . . 11
2.5 Le regole fondamentali del C . . . 12
2.6 I commenti . . . 13
2.7 Cosa sono le librerie . . . 14
2.8 Schema mentale da portarsi dietro . . . 14
2.9 Esercizio . . . 15

3 Variabili e tipi **16**
3.1 Cos'è una variabile . . . 16
3.2 Dichiarazione e assegnazione . . . 16
3.3 I tipi di dato . . . 17
3.3.1 int: numeri interi . . . 17
3.3.2 float e double: numeri decimali . . . 17
3.3.3 char: un singolo carattere . . . 18
3.4 Tabella riassuntiva . . . 18
3.5 Le costanti . . . 18
3.6 I nomi delle variabili . . . 19
3.7 Stampare le variabili con printf . . . 19
3.8 Leggere le variabili con scanf . . . 20
3.9 Esempio completo . . . 21
3.10 Schema mentale . . . 22
3.11 Esercizi . . . 22

4 Operatori **24**
4.1 Cosa sono gli operatori . . . 24
4.2 Operatori aritmetici . . . 24
4.2.1 Il modulo % . . . 25

4.3 Operatori di assegnazione . . . 26
4.3.1 Incremento e decremento . . . 26
4.4 Operatori di confronto . . . 27
4.5 Operatori logici . . . 28
4.5.1 Tabella della verità . . . 28
4.6 Precedenza degli operatori . . . 29
4.7 Il casting . . . 29
4.8 Esempio completo . . . 30
4.9 Schema mentale . . . 31
4.10 Esercizi . . . 31

5 Input e output **33**
5.1 Comunicare con l'utente . . . 33
5.2 printf — stampare sullo schermo . . . 33
5.2.1 I segnaposto . . . 33
5.2.2 Formattare i numeri decimali . . . 34
5.2.3 I caratteri speciali . . . 34
5.2.4 Stampare più valori . . . 35
5.3 scanf — leggere dalla tastiera . . . 35
5.3.1 Leggere tipi diversi . . . 35
5.3.2 Leggere più valori con un solo scanf . . . 36
5.4 Il buffer di input . . . 36
5.5 Allineare l'output . . . 37
5.6 Esempio completo: scheda personaggio . . . 37
5.7 Schema mentale . . . 38
5.8 Esercizi . . . 39

6 Selezione **40**
6.1 Prendere decisioni . . . 40
6.2 if — se . . . 40
6.3 if-else — se… altrimenti . . . 41
6.4 if-else if-else — catena di condizioni . . . 41
6.5 Le graffe sono obbligatorie? . . . 42
6.6 Condizioni composte . . . 43
6.7 if annidati . . . 43
6.8 switch — selezionare tra casi precisi . . . 44
6.8.1 switch vs if-else if . . . 45
6.9 L'operatore ternario . . . 46
6.10 Esempio completo: classificatore di temperature . . . 46
6.11 Schema mentale . . . 47
6.12 Esercizi . . . 48

7 Iterazione **49**
7.1 Ripetere le istruzioni . . . 49
7.2 for — quando sai quante volte ripetere . . . 49
7.2.1 Varianti comuni del for . . . 50
7.3 while — quando non sai quante volte ripetere . . . 51
7.3.1 while per leggere finché l'utente vuole . . . 51
7.4 do-while — eseguire almeno una volta . . . 53
7.5 Confronto tra i tre cicli . . . 54
7.6 break e continue . . . 54
7.6.1 break — esci dal ciclo . . . 54
7.6.2 continue — salta alla prossima iterazione . . . 54
7.7 Cicli annidati . . . 55
7.8 Esempio completo: indovina il numero . . . 56
7.9 Schema mentale . . . 56
7.10 Esercizi . . . 57

8 Funzioni **58**
8.1 Perché le funzioni . . . 58
8.2 Il ruolo del main . . . 58
8.3 Struttura di una funzione . . . 59
8.4 Funzioni che restituiscono un valore . . . 59
8.5 Funzioni void . . . 60
8.6 Il prototipo . . . 61
8.7 Parametri: passaggio per valore . . . 62
8.8 Variabili locali e scope . . . 63
8.9 Strategia per scrivere una funzione . . . 63
8.10 Esempio completo . . . 64
8.11 Schema mentale . . . 66
8.12 Esercizi . . . 66

9 Il tipo booleano **67**
9.1 Vero e falso in C . . . 67
9.2 La libreria stdbool.h . . . 67
9.3 bool, true e false . . . 68
9.4 Usare bool come flag . . . 69
9.5 bool nelle funzioni . . . 70
9.6 bool o int? . . . 71
9.7 Schema mentale . . . 71
9.8 Esercizio . . . 72

II Strutture dati 73

10 Array 74
10.1 Blocco 1: Dichiarare e accedere a un array . . . 74
10.1.1 Dichiarazione . . . 74
10.1.2 Accesso agli elementi . . . 74
10.1.3 Inizializzazione . . . 75
10.1.4 Usare #define per la dimensione . . . 75
10.2 Blocco 2: Saper scorrere un array . . . 76
10.2.1 Leggere un array dall'utente . . . 76
10.2.2 Stampare un array . . . 76
10.2.3 Calcolare la somma . . . 77
10.2.4 Trovare il massimo . . . 77
10.3 Blocco 3: Come si scrive una funzione che lavora sugli array . . . 77
10.3.1 Il tipo restituito . . . 78
10.3.2 I parametri . . . 78
10.3.3 Schema tipico di una funzione sugli array . . . 78
10.3.4 Funzione che conta qualcosa . . . 79
10.3.5 Funzione che trova un valore . . . 79
10.3.6 Funzione che cerca un elemento . . . 79
10.3.7 Funzione che modifica un array . . . 80
10.4 Blocco 4: Array bidimensionali . . . 81
10.4.1 Scorrere una matrice con due cicli annidati . . . 81
10.4.2 Leggere e stampare una matrice . . . 81
10.5 Blocco 5: Strategie per risolvere gli esercizi sugli array . . . 82
10.5.1 Esercizi di tipo contare . . . 82
10.5.2 Esercizi di tipo trovare . . . 83
10.5.3 Esercizi di tipo cercare . . . 83
10.5.4 Esercizi di tipo trasformare . . . 84
10.6 Esercizio — Contare . . . 84
10.7 Esercizio — Trovare . . . 85
10.8 Esercizio — Cercare . . . 86

11 Matrici 88
11.1 Idea chiave . . . 88
11.2 Blocco 1: Dichiarare e inizializzare una matrice . . . 88
11.2.1 Dichiarazione . . . 88
11.2.2 Inizializzazione . . . 89
11.2.3 Accesso a un elemento . . . 89
11.3 Blocco 2: Saper scorrere una matrice . . . 90
11.3.1 Leggere una matrice dall'utente . . . 90
11.3.2 Stampare una matrice . . . 90
11.3.3 Somma di tutti gli elementi . . . 91

11.3.4 Ricerca del massimo . . . 91
11.3.5 Somma di una riga specifica . . . 91
11.3.6 Somma di una colonna specifica . . . 91
11.3.7 Diagonale principale (solo matrici quadrate) . . . 92
11.4 Blocco 3: Funzioni con le matrici . . . 92
11.4.1 Schema delle funzioni più comuni . . . 93
11.5 Blocco 4: sizeof e matrici . . . 93
11.6 Esempio completo . . . 94
11.7 Schema mentale . . . 96
11.8 Esercizi . . . 96

12 Stringhe **98**
12.1 Idea chiave . . . 98
12.2 Perché non si usa scanf per leggere le stringhe . . . 99
12.3 Come funziona fgets . . . 99
12.3.1 fgets protegge dall'overflow . . . 100
12.3.2 Usare sizeof per la dimensione . . . 100
12.3.3 Confronto scanf vs fgets . . . 100
12.4 Blocco 1: Saper leggere correttamente una stringa . . . 101
12.4.1 Quando NON è necessario togliere `\n` . . . 101
12.4.2 Quando invece bisogna togliere `\n` . . . 102
12.5 Blocco 2: Saper scorrere tutta la stringa . . . 102
12.6 Blocco 3: Saper riconoscere il tipo di carattere . . . 102
12.7 Blocco 4: Le funzioni della libreria string.h . . . 104
12.7.1 strcmp — confrontare due stringhe . . . 104
12.8 Blocco 5: Come si scrive una funzione che lavora sulle stringhe . . . 104
12.8.1 Le tre cose da decidere . . . 105
12.8.2 Funzione che conta qualcosa . . . 106
12.8.3 Funzione che trasforma una stringa . . . 106
12.8.4 Funzione che confronta due stringhe . . . 106
12.8.5 Il ruolo del main . . . 107
12.9 Blocco 6: Strategie per risolvere gli esercizi sulle stringhe . . . 107
12.9.1 Esercizi di tipo contare . . . 107
12.9.2 Esercizi di tipo trasformare . . . 108
12.9.3 Esercizi di tipo cercare . . . 108
12.9.4 Esercizi di tipo convertire . . . 108
12.10 Esercizio — Contare . . . 109
12.11 Esercizio — Trasformare . . . 109
12.12 Esercizio — Cercare . . . 110
12.13 Esercizio — Convertire . . . 111

III Puntatori 113

13 Puntatori 114
13.1 Idea chiave . . . 114
13.2 Blocco 1: Capire cos'è un puntatore . . . 114
13.3 Blocco 2: Saper leggere questa scrittura . . . 115
13.3.1 Modificare una variabile tramite puntatore . . . 115
13.4 Blocco 3: Operatori fondamentali . . . 116
13.4.1 Operatore `&` — prendi l'indirizzo . . . 116
13.4.2 Operatore `*` — due significati diversi . . . 116
13.4.3 Esempio completo . . . 116
13.5 Blocco 4: Errori da non fare . . . 117
13.5.1 Errore 1: usare un puntatore non inizializzato . . . 117
13.5.2 Errore 2: confondere indirizzo e valore . . . 117
13.5.3 Errore 3: tipo non compatibile . . . 118
13.6 Blocco 5: Il legame tra puntatori e funzioni . . . 118
13.6.1 Come modificare davvero una variabile . . . 118
13.7 Blocco 6: Schema tipico di funzione con puntatore . . . 119
13.7.1 Caso 1: modificare una variabile . . . 119
13.7.2 Caso 2: scambiare due variabili . . . 119
13.8 Blocco 7: Puntatori e array . . . 120
13.8.1 Equivalenze fondamentali . . . 120
13.8.2 Scorrere un array con un puntatore . . . 120
13.9 Schema mentale . . . 121
13.10 Esercizi . . . 121
13.10.1 Esercizio 1: Primo contatto . . . 121
13.10.2 Esercizio 2: Modifica tramite puntatore . . . 122
13.10.3 Esercizio 3: Funzione che azzera . . . 122
13.10.4 Esercizio 4: Raddoppio . . . 123
13.10.5 Esercizio 5: Scambio di due variabili . . . 123
13.10.6 Esercizio 6: Puntatore e array . . . 124
13.10.7 Esercizio 7: Scorrere un array con puntatore . . . 124
13.10.8 Esercizio 8: Somma con puntatore . . . 125
13.10.9 Esercizio 9: Indice vs puntatore . . . 125
13.10.10 Esercizio 10: Massimo con puntatore . . . 126
13.10.11 Esercizio 11: Restituire un puntatore . . . 126

1 Learning Stack — C

Appunti di programmazione in C

Part I

Fondamenti

2 Introduzione al linguaggio C

2.1 Cos'è un programma

Un programma è un insieme di istruzioni che il computer esegue una dopo l'altra. Tu scrivi le istruzioni, il computer le esegue.

Il problema è che il computer non capisce il linguaggio naturale (come l'italiano), capisce solo sequenze di numeri chiamate **codice macchina**. Noi però non scriviamo in codice macchina: scriviamo in un linguaggio più leggibile, come il C, e poi lo facciamo tradurre.

2.2 Il ciclo di vita di un programma C

Scrivere ed eseguire un programma C richiede tre passi:

```
1. Scrivi il codice sorgente   →   file .c
2. Compila                     →   file eseguibile
3. Esegui
```

Passo 1 — scrivi il codice sorgente Il codice sorgente è il testo che scrivi tu — le istruzioni in linguaggio C, salvate in un file con estensione `.c`.

Passo 2 — compila Il **compilatore** è un programma che traduce il tuo codice sorgente in un file eseguibile che il computer sa eseguire. In C usiamo il compilatore `gcc`.

Passo 3 — esegui Lanci il programma e il computer esegue le istruzioni.

> **Cosa succede se c'è un errore?**
>
> Se il codice ha errori di sintassi, il compilatore si ferma e ti segnala il problema — il file eseguibile non viene creato. Devi correggere l'errore e ricompilare.

> Un programma che **compila** non significa che funziona correttamente — significa solo che la sintassi è corretta. Gli errori di logica si scoprono solo eseguendo il programma.

2.3 Il primo programma

Ogni programma C, anche il più semplice, ha questa struttura:

```
#include <stdio.h>

int main() {
    printf("Ciao, mondo!\n");
    return 0;
}
```

Analizziamolo riga per riga.

`#include <stdio.h>` Dice al compilatore di includere la libreria `stdio.h` — la libreria standard per l'input e l'output. Senza questa riga non puoi usare `printf` e `scanf`. Va sempre messa all'inizio.

`int main()` Ogni programma C ha un punto di partenza: il blocco `main`. Il computer inizia sempre da qui. `int` indica che `main` restituisce un numero intero alla fine.

`{ ... }` Le graffe delimitano il corpo del `main` — tutto il codice che vuoi eseguire va dentro di esse.

`printf("Ciao, mondo!\n");` Stampa il testo sullo schermo. `\n` è il carattere di a capo — senza di esso il cursore resta sulla stessa riga. Il punto e virgola `;` termina ogni istruzione.

`return 0;` Segnala al sistema operativo che il programma è terminato correttamente. 0 significa "nessun errore".

2.4 Come compilare ed eseguire

Apri il terminale e posizionati nella cartella dove hai salvato il file.

```
# Compila
gcc programma.c -o programma

# Esegui
./programma
```

`gcc` è il compilatore. `programma.c` è il tuo file sorgente. `-o programma` dice al compilatore di chiamare il file eseguibile `programma`. Se non metti `-o nome`, il compilatore crea un file chiamato `a.out`.

> **Errori comuni di compilazione**
>
> Il compilatore segnala gli errori con riga e messaggio. Impara a leggere questi messaggi — sono il tuo strumento principale per correggere il codice.
>
> `programma.c:4:5: error: expected ';' before 'return'`
>
> Questo dice: nel file `programma.c`, alla riga 4, colonna 5, manca un punto e virgola prima di `return`.

2.5 Le regole fondamentali del C

Prima di andare avanti, ci sono alcune regole che devi sempre rispettare.

Ogni istruzione termina con ;

```
printf("Ciao\n");   // corretto
printf("Ciao\n")    // ERRORE - manca il punto e virgola
```

Il C distingue maiuscole e minuscole

```
int x = 5;    // corretto
Int x = 5;    // ERRORE - Int non esiste, si scrive int
Printf("..."); // ERRORE - Printf non esiste, si scrive printf
```

Le graffe devono sempre essere aperte e chiuse

```c
int main() {
    printf("Ciao\n");
}   // la graffa chiusa corrisponde a quella aperta
```

Le istruzioni si eseguono in ordine, dall'alto verso il basso

```c
printf("Prima riga\n");
printf("Seconda riga\n");
printf("Terza riga\n");
```

Il computer esegue la prima, poi la seconda, poi la terza — sempre in quest'ordine.

2.6 I commenti

I commenti sono note che scrivi nel codice per spiegare cosa fa — il compilatore li ignora completamente.

```c
// Questo è un commento su una riga sola

/* Questo è un commento
   su più righe */

int main() {
    printf("Ciao\n");  // stampa il saluto
    return 0;
}
```

Usare i commenti è una buona abitudine — rende il codice più leggibile, sia per chi lo legge sia per te quando lo rileggi dopo qualche giorno.

2.7 Cosa sono le librerie

Il C di base sa fare pochissimo. Quasi tutto quello che ti serve — stampare, leggere, calcolare, gestire file — sta dentro delle **librerie**.

Una libreria è una raccolta di funzioni già scritte che puoi usare nel tuo programma. Per usarla devi includerla con `#include`.

Le librerie più comuni che userai:

Libreria	Cosa contiene
`stdio.h`	`printf`, `scanf`, gestione file
`stdlib.h`	`exit`, conversioni, numeri casuali
`string.h`	funzioni per le stringhe
`math.h`	funzioni matematiche (`sqrt`, `pow`…)
`ctype.h`	funzioni sui caratteri (`isupper`, `islower`…)

2.8 Schema mentale da portarsi dietro

Ogni volta che scrivi un programma C, la struttura è sempre questa:

```c
#include <stdio.h>      /* librerie necessarie */

int main() {
    /* 1. dichiara le variabili */
    /* 2. leggi i dati (se servono) */
    /* 3. elabora */
    /* 4. stampa il risultato */
    return 0;
}
```

Non devi memorizzare tutto subito. L'obiettivo di questa prima lezione è capire il ciclo: **scrivi → compila → esegui**.

2.9 Esercizio

Scrivi un programma che stampa le seguenti righe:

```
Benvenuto nel corso di informatica
Linguaggio: C
```

Suggerimenti: - ogni riga vuole un `printf` separato - non dimenticare `\n` alla fine di ogni riga - compila con `gcc` e verifica l'output

3 Variabili e tipi

3.1 Cos'è una variabile

Un programma ha bisogno di ricordare dei dati mentre lavora: il nome di un utente, il risultato di un calcolo, l'età di una persona. Per farlo usa le **variabili**.

Una variabile è una zona di memoria con un nome. Quando dichiari una variabile, stai dicendo al computer: *"tieni da parte un po' di memoria e chiamala con questo nome"*.

```
int eta;
```

Questa riga dichiara una variabile chiamata `eta` che può contenere un numero intero. Per ora è vuota — non ha ancora un valore.

3.2 Dichiarazione e assegnazione

In C devi sempre **dichiarare** una variabile prima di usarla. La dichiarazione dice due cose: il tipo e il nome.

```
int eta;            /* dichiarazione */
eta = 17;           /* assegnazione */
```

Puoi anche fare le due cose insieme:

```
int eta = 17;       /* dichiarazione + inizializzazione */
```

! Non usare una variabile senza inizializzarla

In C, una variabile dichiarata ma non inizializzata contiene un valore casuale — quello che si trovava in quella zona di memoria prima. Usarla porta a risultati imprevedibili.

```
int x;
printf("%d\n", x);  /* stampa un valore casuale - comportamento non definito */
```

Inizializza sempre le variabili prima di usarle.

3.3 I tipi di dato

Il tipo di una variabile determina: - che **tipo di valore** può contenere (numero intero, numero decimale, carattere…) - quanta **memoria** occupa

I tipi fondamentali del C sono quattro.

3.3.1 int: numeri interi

```
int punteggio = 1500;
int livello = 3;
int temperatura = -5;
```

Contiene numeri interi, positivi o negativi. Occupa tipicamente 4 byte.

3.3.2 float e double: numeri decimali

```
float media = 7.5;
double pigreco = 3.14159265358979;
```

`float` è a precisione singola (4 byte), `double` a precisione doppia (8 byte). Nella pratica si usa quasi sempre `double` perché è più preciso.

3.3.3 char: un singolo carattere

```
char iniziale = 'A';
char risposta = 's';
```

Contiene un singolo carattere. Si scrive sempre tra **apici singoli** `' '` — non tra virgolette. Occupa 1 byte.

> **char e numeri**
>
> In C i caratteri sono in realtà numeri — ogni carattere ha un codice numerico secondo la tabella ASCII. `'A'` corrisponde al numero 65, `'a'` al numero 97, `'0'` al numero 48. Questo significa che puoi fare operazioni aritmetiche sui caratteri:
>
> ```
> char c = 'A';
> printf("%c\n", c + 1);  /* stampa 'B' */
> ```
>
> Questa proprietà è fondamentale quando lavori con le stringhe.

3.4 Tabella riassuntiva

Tipo	Cosa contiene	Esempio
`int`	numero intero	`42`, `-7`, `0`
`float`	numero decimale (precisione singola)	`3.14`
`double`	numero decimale (precisione doppia)	`3.14159265`
`char`	singolo carattere	`'A'`, `'z'`, `'3'`

3.5 Le costanti

A volte hai un valore che non deve mai cambiare — per esempio il numero di giorni in una settimana o il valore del pi greco. In quel caso usi una **costante**.

```
#define PI 3.14159
#define GIORNI_SETTIMANA 7
#define DIMENSIONE 100
```

`#define` non è una variabile — è una sostituzione: ogni volta che il compilatore trova `PI` nel codice, lo sostituisce con `3.14159`. Per convenzione i nomi delle costanti si scrivono in maiuscolo.

3.6 I nomi delle variabili

Puoi chiamare le variabili come vuoi, ma ci sono delle regole:

Regole obbligatorie: - può contenere lettere, cifre e underscore `_` - non può iniziare con una cifra - non può essere una parola riservata del C (`int`, `if`, `while`…)

Convenzioni (buone abitudini): - usa nomi significativi: `eta` è meglio di `x`, `punteggioGiocatore` è meglio di `pg` - usa il camelCase per nomi composti: `punteggioTotale`, `nomeUtente` - evita nomi di una sola lettera (tranne `i`, `j`, `k` nei cicli)

```
int eta = 17;              /* buono */
int e = 17;                /* scarso - cosa vuol dire e? */
int punteggio_totale = 0;  /* accettabile */
int punteggioTotale = 0;   /* preferibile */
int 2giocatori = 0;        /* ERRORE - inizia con un numero */
int int = 0;               /* ERRORE - int è una parola riservata */
```

3.7 Stampare le variabili con printf

Per stampare una variabile con `printf` devi usare un **segnaposto** — un codice che indica il tipo della variabile.

Tipo	Segnaposto
`int`	`%d`
`float`	`%f`
`double`	`%lf` oppure `%f`

Tipo	Segnaposto
`char`	`%c`
stringa	`%s`

```
int eta = 17;
double media = 8.5;
char voto = 'A';

printf("Età: %d\n", eta);
printf("Media: %.2f\n", media);   /* .2 = 2 cifre decimali */
printf("Voto: %c\n", voto);
```

Output:

```
Età: 17
Media: 8.50
Voto: A
```

Puoi stampare più variabili nella stessa `printf`:

```
printf("Età: %d, media: %.1f\n", eta, media);
```

I segnaposto vengono sostituiti nell'ordine in cui compaiono — il primo `%d` con il primo valore, il secondo con il secondo, e così via.

3.8 Leggere le variabili con scanf

Per leggere un valore inserito dall'utente si usa `scanf`.

```
int eta;
printf("Inserisci la tua età: ");
scanf("%d", &eta);
printf("Hai %d anni.\n", eta);
```

La `&` davanti al nome della variabile è obbligatoria — dice a `scanf` dove in memoria salvare il valore letto. Per ora ricordala come una regola: con `scanf` si mette sempre `&`.

```c
int n;
double x;
char c;

scanf("%d", &n);     /* legge un intero */
scanf("%lf", &x);    /* legge un double */
scanf(" %c", &c);    /* legge un carattere - nota lo spazio prima di %c */
```

> ⚠ Lo spazio prima di `%c`
>
> Quando leggi un carattere con `scanf`, metti sempre uno spazio prima di `%c`:
>
> ```c
> scanf(" %c", &c);
> ```
>
> Senza lo spazio, `scanf` potrebbe leggere il carattere di invio rimasto nel buffer dalla lettura precedente invece del carattere che vuoi leggere.

3.9 Esempio completo

```c
#include <stdio.h>

int main() {
    /* Dichiarazione delle variabili */
    int eta;
    double altezza;
    char iniziale;

    /* Lettura dei dati */
    printf("Inserisci la tua età: ");
    scanf("%d", &eta);

    printf("Inserisci la tua altezza in metri (es. 1.75): ");
    scanf("%lf", &altezza);

    printf("Inserisci l'iniziale del tuo nome: ");
    scanf(" %c", &iniziale);
```

```c
    /* Stampa dei risultati */
    printf("\n--- RIEPILOGO ---\n");
    printf("Iniziale: %c\n", iniziale);
    printf("Età:      %d anni\n", eta);
    printf("Altezza:  %.2f m\n", altezza);

    return 0;
}
```

3.10 Schema mentale

Quando dichiari una variabile, chiediti sempre tre cose:

```
1. Che tipo di dato devo memorizzare?
   → numero intero?    usa int
   → numero decimale?  usa double
   → singolo carattere? usa char

2. Che nome le do?
   → significativo, che descriva il contenuto

3. Ha già un valore oppure lo leggo dall'utente?
   → ce l'ha già → inizializzala subito
   → lo leggo    → dichiara, poi usa scanf
```

3.11 Esercizi

Esercizio 1 Scrivi un programma che: - dichiara una variabile `int` per l'anno di nascita e una `double` per la media scolastica - legge entrambi i valori dall'utente - stampa: `"Sei nato nel ANNO e hai una media di MEDIA"`

Esercizio 2 Scrivi un programma che legge due numeri interi e stampa la loro somma, differenza, prodotto e quoziente.

Esercizio 3 Scrivi un programma che legge un carattere e stampa il suo valore numerico ASCII.

`Esempio: inserisci 'A' → stampa 65`

Suggerimento: puoi stampare un **`char`** *sia con* **`%c`** *(come carattere) sia con* **`%d`** *(come numero).*

4 Operatori

4.1 Cosa sono gli operatori

Un operatore è un simbolo che esegue un'operazione su uno o più valori. Li conosci già dalla matematica: `+`, `-`, `*`, `/`. In C ce ne sono altri, specifici della programmazione.

Gli operatori si dividono in tre famiglie: - **aritmetici** — fanno calcoli - **relazionali** — confrontano due valori - **logici** — combinano condizioni

---

4.2 Operatori aritmetici

Operatore	Significato	Esempio	Risultato
`+`	addizione	`5 + 3`	`8`
`-`	sottrazione	`5 - 3`	`2`
`*`	moltiplicazione	`5 * 3`	`15`
`/`	divisione	`5 / 2`	`2`
`%`	modulo (resto)	`5 % 2`	`1`

```c
int a = 10;
int b = 3;

printf("%d\n", a + b);   /* 13 */
printf("%d\n", a - b);   /* 7  */
printf("%d\n", a * b);   /* 30 */
printf("%d\n", a / b);   /* 3  - divisione intera! */
printf("%d\n", a % b);   /* 1  - resto della divisione */
```

! La divisione intera

Quando dividi due `int`, il risultato è un `int` — la parte decimale viene troncata, non arrotondata.

```
int a = 5;
int b = 2;
printf("%d\n", a / b);   /* stampa 2, non 2.5 */
```

Se vuoi il risultato decimale, almeno uno dei due deve essere `double`:

```
double risultato = 5.0 / 2;    /* 2.5 */
double risultato = (double)5 / 2;  /* 2.5 - casting */
```

4.2.1 Il modulo %

Il modulo restituisce il **resto** della divisione intera. È uno degli operatori più utili in programmazione.

```
10 % 3   /* 1  - perché 10 = 3*3 + 1 */
12 % 4   /* 0  - perché 12 = 4*3 + 0 */
7  % 2   /* 1  - perché 7  = 2*3 + 1 */
```

A cosa serve il modulo? A molte cose:

```
/* Verificare se un numero è pari o dispari */
if (n % 2 == 0)   /* n è pari  */
if (n % 2 != 0)   /* n è dispari */

/* Estrarre l'ultima cifra di un numero */
int ultima = n % 10;   /* es: 347 % 10 = 7 */

/* Tenere un indice dentro un range */
int i = posizione % dimensione;
```

4.3 Operatori di assegnazione

L'operatore di assegnazione base è `=`. Ma ci sono anche forme abbreviate molto usate:

Operatore	Equivale a	Esempio
`=`	assegna	`x = 5`
`+=`	`x = x + n`	`x += 3`
`-=`	`x = x - n`	`x -= 3`
`*=`	`x = x * n`	`x *= 2`
`/=`	`x = x / n`	`x /= 2`
`%=`	`x = x % n`	`x %= 2`

```
int punteggio = 100;
punteggio += 50;    /* punteggio diventa 150 */
punteggio -= 20;    /* punteggio diventa 130 */
punteggio *= 2;     /* punteggio diventa 260 */
```

4.3.1 Incremento e decremento

Due operatori molto usati nei cicli:

```
int i = 0;
i++;    /* equivale a i = i + 1 */
i--;    /* equivale a i = i - 1 */
```

```
/* Questi quattro sono tutti equivalenti */
i = i + 1;
i += 1;
i++;
++i;
```

`i++` vs `++i`

La differenza emerge solo quando li usi dentro un'espressione:

```
int i = 5;
int a = i++;   /* a = 5, poi i diventa 6 - prima assegna, poi incrementa */
int b = ++i;   /* i diventa 7, poi b = 7 - prima incrementa, poi assegna */
```

Nei cicli, dove li usi da soli, non fa differenza. Usa la forma che preferisci — `i++` è più comune.

4.4 Operatori di confronto

Gli operatori relazionali **confrontano** due valori e restituiscono `1` (vero) oppure `0` (falso).

Operatore	Significato	Esempio	Risultato
`==`	uguale a	`5 == 5`	`1` (vero)
`!=`	diverso da	`5 != 3`	`1` (vero)
`<`	minore di	`3 < 5`	`1` (vero)
`>`	maggiore di	`3 > 5`	`0` (falso)
`<=`	minore o uguale	`5 <= 5`	`1` (vero)
`>=`	maggiore o uguale	`3 >= 5`	`0` (falso)

```c
int a = 10;
int b = 20;

printf("%d\n", a == b);   /* 0 - falso  */
printf("%d\n", a != b);   /* 1 - vero   */
printf("%d\n", a < b);    /* 1 - vero   */
printf("%d\n", a > b);    /* 0 - falso  */
```

`=` e `==` sono cose completamente diverse

`=` è l'assegnazione — modifica il valore di una variabile. `==` è il confronto — verifica se due valori sono uguali.
Questo è uno degli errori più comuni in C:

```c
if (x = 5)    /* SBAGLIATO - assegna 5 a x, poi valuta x */
if (x == 5)   /* CORRETTO  - confronta x con 5 */
```

Il primo non dà errore di compilazione ma si comporta in modo inatteso — stai assegnando, non confrontando.

4.5 Operatori logici

Gli operatori logici combinano più condizioni.

Operatore	Significato	Esempio
`&&`	AND — entrambe vere	`a > 0 && a < 10`
`||`	OR — almeno una vera	`a < 0 || a > 100`
`!`	NOT — nega la condizione	`!(a == 0)`

```c
int voto = 7;

/* AND - il voto deve essere tra 6 e 10 compresi */
if (voto >= 6 && voto <= 10) {
    printf("Promosso\n");
}

/* OR - il voto è insufficiente o fuori scala */
if (voto < 6 || voto > 10) {
    printf("Valore non valido\n");
}

/* NOT - il voto NON è sufficiente */
if (!(voto >= 6)) {
    printf("Insufficiente\n");
}
```

4.5.1 Tabella della verità

a	b	a && b	a || b	!a
0 (falso)	0 (falso)	0	0	1
0 (falso)	1 (vero)	0	1	1
1 (vero)	0 (falso)	0	1	0
1 (vero)	1 (vero)	1	1	0

4.6 Precedenza degli operatori

Quando un'espressione contiene più operatori, il C segue un ordine preciso — come in matematica dove la moltiplicazione viene prima dell'addizione.

```
1. ! (NOT)
2. * / % (moltiplicazione, divisione, modulo)
3. + - (addizione, sottrazione)
4. < > <= >= (confronto)
5. == != (uguaglianza)
6. && (AND)
7. || (OR)
8. = += -= ... (assegnazione)
```

```c
int x = 2 + 3 * 4;      /* 14, non 20 - prima 3*4, poi +2 */
int y = (2 + 3) * 4;    /* 20 - le parentesi cambiano l'ordine */

int ok = a > 0 && b < 10;  /* prima valuta a>0 e b<10, poi && */
```

Usa le parentesi

Non devi memorizzare tutta la tabella di precedenza. Quando hai dubbi, usa le parentesi — rendono il codice più chiaro e prevengono errori.

```c
/* Questi due si leggono diversamente */
if (a == b && c == d || e == f)    /* difficile da leggere */
if ((a == b && c == d) || e == f)  /* molto più chiaro */
```

4.7 Il casting

A volte devi convertire un tipo in un altro. In C si usa il **casting** con le parentesi tonde:

```c
int a = 5;
int b = 2;

/* Senza casting: divisione intera */
double risultato1 = a / b;           /* 2.0 - sbagliato! */
```

```
/* Con casting: divisione decimale */
double risultato2 = (double)a / b;   /* 2.5 - corretto */
```

Il casting `(double)a` dice al compilatore: *"tratta `a` come se fosse un `double` per questa operazione"*. Il valore originale di `a` non cambia.

```
/* Altri esempi di casting */
int x = (int)3.9;      /* x = 3 - la parte decimale viene troncata */
char c = (char)65;     /* c = 'A' - il numero 65 diventa il carattere A */
```

4.8 Esempio completo

```
#include <stdio.h>

int main() {
    int a, b;

    printf("Inserisci due numeri interi: ");
    scanf("%d %d", &a, &b);

    printf("\n--- OPERAZIONI ---\n");
    printf("Somma:       %d\n", a + b);
    printf("Differenza:  %d\n", a - b);
    printf("Prodotto:    %d\n", a * b);

    if (b != 0) {
        printf("Quoziente:   %d\n", a / b);
        printf("Resto:       %d\n", a % b);
        printf("Divisione:   %.2f\n", (double)a / b);
    } else {
        printf("Divisione impossibile: b è zero\n");
    }

    printf("\n--- CONFRONTI ---\n");
    printf("a == b?  %d\n", a == b);
    printf("a != b?  %d\n", a != b);
```

```c
    printf("a < b?   %d\n", a < b);
    printf("a > b?   %d\n", a > b);

    printf("\n--- PARI O DISPARI ---\n");
    printf("a è %s\n", (a % 2 == 0) ? "pari" : "dispari");
    printf("b è %s\n", (b % 2 == 0) ? "pari" : "dispari");

    return 0;
}
```

4.9 Schema mentale

```
Devo fare un calcolo?
    → usa + - * /
    → attento alla divisione intera tra int

Devo confrontare due valori?
    → usa == != < > <= >=
    → non confondere = (assegna) con == (confronta)

Devo combinare più condizioni?
    → entrambe vere      → &&
    → almeno una vera    → ||
    → negare             → !

Ho dubbi sulla precedenza?
    → usa le parentesi
```

4.10 Esercizi

Esercizio 1 Scrivi un programma che legge un numero intero e stampa: - se è pari o dispari - se è positivo, negativo o zero - le sue ultime due cifre (suggerimento: usa `%`)

Esercizio 2 Scrivi un programma che legge due numeri interi `a` e `b` e stampa quante volte `b` entra in `a` e quanto rimane (divisione con resto).

```
Esempio: a=17, b=5 → 17 = 5 * 3 + 2
```

Esercizio 3 Scrivi un programma che legge tre voti (interi da 0 a 10) e stampa la media con due decimali. Usa il casting per ottenere una divisione decimale.

5 Input e output

5.1 Comunicare con l'utente

Un programma utile deve saper fare due cose: **ricevere dati** dall'utente e **restituire risultati**. In C questo si fa con due funzioni della libreria `stdio.h`:

- `printf` — stampa dati sullo schermo
- `scanf` — legge dati dalla tastiera

Le hai già incontrate nelle lezioni precedenti. Ora le vediamo in modo completo.

5.2 printf — stampare sullo schermo

`printf` stampa testo e valori sullo schermo. La struttura è sempre:

```
printf("stringa di formato", variabile1, variabile2, ...);
```

La **stringa di formato** è il testo da stampare. Può contenere testo normale e **segnaposto** — codici che indicano dove inserire i valori delle variabili.

```
int eta = 17;
printf("Ho %d anni.\n", eta);
/* stampa: Ho 17 anni. */
```

5.2.1 I segnaposto

Segnaposto	Tipo	Esempio
`%d`	`int`	`printf("%d", 42)` → `42`
`%f`	`float` / `double`	`printf("%f", 3.14)` → `3.140000`
`%lf`	`double` (con scanf)	—
`%c`	`char`	`printf("%c", 'A')` → `A`
`%s`	stringa	`printf("%s", "ciao")` → `ciao`

5.2.2 Formattare i numeri decimali

Di default `%f` stampa 6 cifre decimali. Puoi controllare quante cifre vuoi:

```
double media = 7.666666;

printf("%f\n",   media);    /* 7.666666 - default */
printf("%.2f\n", media);    /* 7.67     - 2 decimali */
printf("%.0f\n", media);    /* 8        - nessun decimale */
printf("%8.2f\n", media);   /* __7.67   - 8 caratteri totali, 2 decimali */
```

5.2.3 I caratteri speciali

All'interno delle stringhe alcuni caratteri hanno un significato speciale:

Sequenza	Effetto
`\n`	vai a capo
`\t`	tabulazione orizzontale
`\\`	stampa il carattere `\`
`\"`	stampa il carattere `"`
`%%`	stampa il carattere `%`

```
printf("Nome:\tArthas\n");
printf("Punteggio:\t%d%%\n", 85);
/* stampa:
   Nome:   Arthas
   Punteggio:  85%  */
```

5.2.4 Stampare più valori

```
int livello = 5;
double vita = 87.5;
char classe = 'G';

printf("Classe: %c | Livello: %d | Vita: %.1f%%\n", classe, livello, vita);
/* stampa: Classe: G | Livello: 5 | Vita: 87.5% */
```

I segnaposto vengono sostituiti nell'ordine: il primo `%c` con `classe`, il primo `%d` con `livello`, e così via.

5.3 scanf — leggere dalla tastiera

`scanf` legge quello che l'utente digita e lo salva in una variabile. La struttura è:

```
scanf("segnaposto", &variabile);
```

La `&` davanti alla variabile è obbligatoria — indica l'indirizzo in memoria dove salvare il valore. Vedremo meglio cosa significa quando studieremo i puntatori. Per ora: **con `scanf` si mette sempre `&`**.

```
int eta;
printf("Inserisci la tua età: ");
scanf("%d", &eta);
```

5.3.1 Leggere tipi diversi

```
int n;
double x;
char c;
char nome[50];   /* stringa - la vediamo dopo */

scanf("%d", &n);      /* legge un intero */
scanf("%lf", &x);     /* legge un double - nota: %lf, non %f */
scanf(" %c", &c);     /* legge un carattere - nota lo spazio prima di %c */
scanf("%s", nome);    /* legge una parola - senza & per le stringhe */
```

⚠ Tre cose da ricordare con scanf

1. Con `scanf` si usa `%lf` per i double, non `%f`.

```
double x;
scanf("%lf", &x);   /* corretto */
scanf("%f", &x);    /* sbagliato - comportamento indefinito */
```

2. Lo spazio prima di `%c` è necessario.

```
int n;
char c;
scanf("%d", &n);
scanf(" %c", &c);   /* lo spazio consuma l'invio rimasto nel buffer */
```

Senza lo spazio, `scanf` leggerebbe il carattere `\n` rimasto dal `%d` precedente.

3. Non mettere `\n` nella stringa di formato di scanf.

```
scanf("%d\n", &n);   /* sbagliato - aspetta un altro invio */
scanf("%d", &n);     /* corretto */
```

5.3.2 Leggere più valori con un solo scanf

```
int a, b;
scanf("%d %d", &a, &b);   /* legge due interi separati da spazio */
```

L'utente può separare i valori con spazi o con invio — `scanf` li gestisce allo stesso modo.

5.4 Il buffer di input

Quando l'utente preme invio, il testo digitato va in una zona di memoria chiamata **buffer di input**. `scanf` legge dal buffer — non direttamente dalla tastiera.

Questo causa un problema classico: quando `scanf` legge un intero con `%d`, lascia nel buffer il carattere `\n` (l'invio). Se subito dopo leggi un carattere con `%c`, `scanf` troverà quel `\n` nel buffer e lo leggerà — invece di aspettare che l'utente digiti qualcosa.

```c
int n;
char c;

scanf("%d", &n);      /* legge il numero, lascia \n nel buffer */
scanf("%c", &c);      /* legge \n invece di aspettare l'utente! */
scanf(" %c", &c);     /* lo spazio consuma il \n - corretto */
```

Schema mentale: **ogni volta che leggi un `char` dopo un `int` o un `double`, metti uno spazio prima di `%c`.**

5.5 Allineare l'output

Quando stampi tabelle o elenchi, puoi allineare i valori con i modificatori di larghezza:

```c
printf("%-15s %5s %8s\n", "Nome", "Voto", "Media");
printf("%-15s %5d %8.2f\n", "Arthas", 8, 7.83);
printf("%-15s %5d %8.2f\n", "Gandalf", 9, 8.50);
printf("%-15s %5d %8.2f\n", "Legolas", 7, 6.67);
```

Output:

```
Nome             Voto    Media
Arthas              8     7.83
Gandalf             9     8.50
Legolas             7     6.67
```

- `%15s` — testo allineato a destra in 15 caratteri
- `%-15s` — testo allineato a sinistra in 15 caratteri (il `-` inverte l'allineamento)
- `%5d` — intero in 5 caratteri, allineato a destra
- `%8.2f` — decimale in 8 caratteri totali con 2 cifre dopo la virgola

---

5.6 Esempio completo: scheda personaggio

```c
#include <stdio.h>

int main() {
    char nome[50];
    int livello;
    double vita;
    char classe;

    /* Lettura dei dati */
    printf("=== CREA IL TUO PERSONAGGIO ===\n\n");

    printf("Nome: ");
    scanf("%s", nome);

    printf("Classe (G=Guerriero, M=Mago, A=Arciere): ");
    scanf(" %c", &classe);

    printf("Livello (1-10): ");
    scanf("%d", &livello);

    printf("Punti vita: ");
    scanf("%lf", &vita);

    /* Stampa della scheda */
    printf("\n--- SCHEDA PERSONAGGIO ---\n");
    printf("%-10s %s\n",   "Nome:",    nome);
    printf("%-10s %c\n",   "Classe:",  classe);
    printf("%-10s %d\n",   "Livello:", livello);
    printf("%-10s %.1f\n", "Vita:",    vita);

    return 0;
}
```

5.7 Schema mentale

```
Devo stampare qualcosa?
    → printf("...", valori)
    → scegli il segnaposto giusto: %d %f %c %s
```

```
    → controlla i decimali: %.2f
    → vai a capo con \n

Devo leggere qualcosa?
    → scanf("segnaposto", &variabile)
    → usa %lf per i double
    → metti spazio prima di %c se hai letto un int prima
    → non mettere \n nella stringa di formato

Devo allineare l'output?
    → usa la larghezza: %10d, %-15s, %8.2f
```

5.8 Esercizi

Esercizio 1 Scrivi un programma che legge nome, cognome e anno di nascita di una persona e stampa:

```
Benvenuto, Mario Rossi!
Sei nato nel 1990.
```

Esercizio 2 Scrivi un programma che legge tre voti interi e stampa una tabella allineata con i voti e la media:

```
Voto 1:     7
Voto 2:     8
Voto 3:     6
Media:   7.00
```

Esercizio 3 Scrivi un programma che legge un intero e un carattere (in quest'ordine) e li stampa entrambi. Fai attenzione al problema del buffer.

6 Selezione

6.1 Prendere decisioni

Fino ad ora i nostri programmi eseguono sempre le stesse istruzioni, dall'alto verso il basso, senza mai cambiare percorso. Ma nella realtà un programma deve saper **prendere decisioni**: fare una cosa se una condizione è vera, un'altra se è falsa.

Le strutture di selezione servono esattamente a questo — permettono al programma di scegliere quale blocco di istruzioni eseguire in base a una condizione.

6.2 if — se

La struttura più semplice è `if`. Esegue un blocco di istruzioni solo se la condizione è vera.

```
if (condizione) {
    /* istruzioni eseguite solo se la condizione è vera */
}
```

```
int voto = 7;

if (voto >= 6) {
    printf("Promosso!\n");
}
```

Se `voto` è 7, la condizione `voto >= 6` è vera e viene stampato `Promosso!`. Se `voto` fosse 4, la condizione sarebbe falsa e `printf` non verrebbe eseguita.

6.3 if-else — se... altrimenti

Quando vuoi fare una cosa se la condizione è vera e un'altra se è falsa, usi `if-else`.

```
if (condizione) {
    /* eseguito se la condizione è vera */
} else {
    /* eseguito se la condizione è falsa */
}
```

```
int voto = 4;

if (voto >= 6) {
    printf("Promosso!\n");
} else {
    printf("Bocciato.\n");
}
```

Uno dei due blocchi viene sempre eseguito — mai entrambi, mai nessuno.

6.4 if-else if-else — catena di condizioni

Quando hai più casi da gestire, puoi concatenare più condizioni con `else if`.

```
if (condizione1) {
    /* eseguito se condizione1 è vera */
} else if (condizione2) {
    /* eseguito se condizione1 è falsa e condizione2 è vera */
} else if (condizione3) {
    /* eseguito se le prime due sono false e condizione3 è vera */
} else {
    /* eseguito se tutte le condizioni sono false */
}
```

```
int voto;
printf("Inserisci il voto: ");
scanf("%d", &voto);
```

```c
if (voto >= 9) {
    printf("Ottimo\n");
} else if (voto >= 7) {
    printf("Buono\n");
} else if (voto >= 6) {
    printf("Sufficiente\n");
} else {
    printf("Insufficiente\n");
}
```

> **Come funziona la catena**
>
> Le condizioni vengono valutate dall'alto verso il basso. Appena una è vera, viene eseguito quel blocco e le altre vengono ignorate — anche se sarebbero vere anch'esse.
> Per questo l'ordine conta: nell'esempio sopra, se il voto è 9, la prima condizione `voto >= 9` è vera e si stampa `Ottimo`. Non viene più controllato se `voto >= 7` — anche se sarebbe vero.
> Scrivi sempre le condizioni dalla più restrittiva alla meno restrittiva.

6.5 Le graffe sono obbligatorie?

Tecnicamente, se il blocco contiene una sola istruzione, le graffe si possono omettere:

```c
if (voto >= 6)
    printf("Promosso!\n");   /* funziona */
```

Ma è una cattiva abitudine. Se aggiungi una seconda istruzione dimenticandoti le graffe, il programma non fa quello che ti aspetti:

```c
if (voto >= 6)
    printf("Promosso!\n");
    printf("Complimenti!\n");  /* viene eseguita SEMPRE, non solo se promosso */
```

! Metti sempre le graffe

Anche quando il blocco ha una sola istruzione, usa sempre le graffe. Il codice è più leggibile e meno soggetto a errori quando lo modifichi.

```
if (voto >= 6) {
    printf("Promosso!\n");
}
```

6.6 Condizioni composte

Puoi combinare più condizioni con gli operatori logici `&&` e `||`.

```
int eta = 17;
int consenso = 1;   /* 1 = sì, 0 = no */

/* AND - entrambe le condizioni devono essere vere */
if (eta >= 18 && consenso == 1) {
    printf("Accesso consentito\n");
}

/* OR - almeno una condizione deve essere vera */
if (eta < 13 || eta > 65) {
    printf("Tariffa ridotta\n");
}

/* Range - il valore deve essere tra due estremi */
if (voto >= 6 && voto <= 10) {
    printf("Voto valido\n");
}
```

6.7 if annidati

Puoi mettere un `if` dentro un altro `if`. Si chiamano **if annidati**.

```c
int voto;
char comportamento;

if (voto >= 6) {
    printf("Promosso in termini di voto.\n");
    if (comportamento == 'O') {   /* O = Ottimo */
        printf("Menzione speciale per il comportamento!\n");
    }
} else {
    printf("Bocciato.\n");
}
```

> **Non annidare troppo**
>
> Più livelli di annidamento rendono il codice difficile da leggere. Se ti trovi con tre o quattro livelli, probabilmente c'è un modo più chiaro di scrivere la stessa logica.
> Regola pratica: oltre due livelli di annidamento, fermati e rifletti.

6.8 switch — selezionare tra casi precisi

Quando devi confrontare una variabile con molti valori precisi, `switch` è più leggibile di una lunga catena di `if-else if`.

```c
switch (variabile) {
    case valore1:
        /* istruzioni */
        break;
    case valore2:
        /* istruzioni */
        break;
    default:
        /* eseguito se nessun caso corrisponde */
        break;
}
```

```c
char classe;
printf("Inserisci la classe (G/M/A): ");
```

```c
scanf(" %c", &classe);

switch (classe) {
    case 'G':
        printf("Sei un Guerriero.\n");
        break;
    case 'M':
        printf("Sei un Mago.\n");
        break;
    case 'A':
        printf("Sei un Arciere.\n");
        break;
    default:
        printf("Classe non riconosciuta.\n");
        break;
}
```

Non dimenticare il break

Senza `break`, dopo aver eseguito un caso il programma continua a eseguire i casi successivi — anche se non corrispondono. Questo si chiama **fall-through** ed è quasi sempre un errore.

```c
switch (voto) {
    case 10:
        printf("Ottimo\n");
        /* manca break - continua nel caso 9! */
    case 9:
        printf("Buono\n");
        break;
}
/* Se voto è 10, stampa sia "Ottimo" che "Buono" */
```

Metti sempre `break` alla fine di ogni caso, incluso `default`.

6.8.1 switch vs if-else if

	`switch`	`if-else if`
Confronta	valori precisi (`==`)	qualsiasi condizione
Tipi	`int`, `char`	qualsiasi tipo

	`switch`	`if-else if`
Leggibilità	molto chiaro con molti casi	preferibile con pochi casi o range

Usa `switch` quando confronti una variabile con valori precisi e hai molti casi. Usa `if-else if` quando le condizioni coinvolgono range, operatori `<`, `>`, o tipi `double`.

6.9 L'operatore ternario

L'operatore ternario è una forma compatta di `if-else` che restituisce un valore.

```
condizione ? valore_se_vera : valore_se_falsa
```

```
int voto = 7;

/* Con if-else */
if (voto >= 6) {
    printf("Promosso\n");
} else {
    printf("Bocciato\n");
}

/* Con operatore ternario - equivalente */
printf("%s\n", voto >= 6 ? "Promosso" : "Bocciato");
```

Usalo solo per casi semplici — quando la condizione e i valori sono brevi e chiari. Per logica complessa, usa `if-else`.

6.10 Esempio completo: classificatore di temperature

```c
#include <stdio.h>

int main() {
    double temperatura;

    printf("Inserisci la temperatura: ");
    scanf("%lf", &temperatura);

    printf("Temperatura: %.1f°C → ", temperatura);

    if (temperatura < 0) {
        printf("Sotto zero - ghiaccio!\n");
    } else if (temperatura < 10) {
        printf("Molto freddo.\n");
    } else if (temperatura < 20) {
        printf("Fresco.\n");
    } else if (temperatura < 30) {
        printf("Ideale.\n");
    } else if (temperatura < 40) {
        printf("Caldo.\n");
    } else {
        printf("Molto caldo!\n");
    }

    /* Avviso separato con condizione composta */
    if (temperatura >= 37.0 && temperatura <= 38.5) {
        printf("Attenzione: possibile febbre.\n");
    }

    return 0;
}
```

6.11 Schema mentale

```
Devo eseguire qualcosa solo se una condizione è vera?
    → if (condizione) { ... }

Devo fare una cosa o l'altra?
```

```
    → if (condizione) { ... } else { ... }

Ho più di due casi?
    → if ... else if ... else if ... else
    → ordina dalla condizione più restrittiva alla meno restrittiva

Confronto un valore con molti valori precisi (int o char)?
    → switch - più leggibile
    → non dimenticare break in ogni caso

Ho una condizione semplice e voglio un valore?
    → operatore ternario: condizione ? a : b
```

6.12 Esercizi

Esercizio 1 Scrivi un programma che legge un numero intero e stampa se è positivo, negativo o zero.

Esercizio 2 Scrivi un programma che legge un voto da 0 a 10 e stampa il giudizio: - 9-10: Ottimo - 7-8: Buono - 6: Sufficiente - 4-5: Insufficiente - 0-3: Gravemente insufficiente - altro: valore non valido

Esercizio 3 Scrivi un programma che legge il numero del mese (1-12) e stampa il nome del mese e il numero di giorni che ha. Usa `switch`. Per febbraio stampa "28 o 29 giorni".

Esercizio 4 Scrivi un programma che legge tre numeri interi e stampa il maggiore tra i tre.

7 Iterazione

7.1 Ripetere le istruzioni

Immagina di dover stampare i numeri da 1 a 100. Scrivere cento `printf` non è una soluzione — è lungo, noioso e impossibile da modificare.

Le strutture di iterazione — chiamate anche **cicli** o **loop** — permettono di ripetere un blocco di istruzioni più volte. In C ce ne sono tre: `for`, `while` e `do-while`.

7.2 for — quando sai quante volte ripetere

Il ciclo `for` è la scelta giusta quando sai esattamente quante volte vuoi ripetere qualcosa.

```
for (inizializzazione; condizione; aggiornamento) {
    /* istruzioni da ripetere */
}
```

Le tre parti dentro le parentesi: - **inizializzazione** — eseguita una volta sola all'inizio - **condizione** — controllata prima di ogni ripetizione; se è falsa il ciclo si ferma - **aggiornamento** — eseguito alla fine di ogni ripetizione

```
int i;
for (i = 1; i <= 5; i++) {
    printf("%d\n", i);
}
```

Output:

```
1
2
3
4
5
```

Passo per passo: 1. `i = 1` — inizializzazione 2. `i <= 5`? Sì → esegui il corpo → `i++` → `i` diventa 2 3. `i <= 5`? Sì → esegui il corpo → `i++` → `i` diventa 3 4. … 5. `i <= 5`? No (i è 6) → il ciclo si ferma

> **La variabile contatore**
>
> La variabile `i` si chiama **contatore** — tiene traccia di quante volte il ciclo ha girato. Per convenzione si usa `i` (poi `j`, `k` per cicli annidati). Deve essere dichiarata prima del `for`.
>
> ```
> int i;
> for (i = 0; i < 10; i++) { ... }   /* da 0 a 9 - 10 ripetizioni */
> for (i = 1; i <= 10; i++) { ... }  /* da 1 a 10 - 10 ripetizioni */
> for (i = 10; i >= 1; i--) { ... }  /* da 10 a 1 - conto alla rovescia */
> ```

7.2.1 Varianti comuni del for

```
int i;

/* Conta da 0 a N-1 - molto usato con gli array */
for (i = 0; i < 10; i++) {
    printf("%d ", i);   /* 0 1 2 3 4 5 6 7 8 9 */
}

/* Conta solo i pari */
for (i = 0; i <= 20; i += 2) {
    printf("%d ", i);   /* 0 2 4 6 8 10 12 14 16 18 20 */
}

/* Conta alla rovescia */
for (i = 5; i >= 1; i--) {
    printf("%d ", i);   /* 5 4 3 2 1 */
}

/* Somma da 1 a N */
```

```c
int somma = 0;
int n = 10;
for (i = 1; i <= n; i++) {
    somma += i;
}
printf("Somma: %d\n", somma);   /* 55 */
```

7.3 while — quando non sai quante volte ripetere

Il ciclo `while` ripete un blocco finché una condizione è vera. È la scelta giusta quando non sai in anticipo quante volte ripetere.

```c
while (condizione) {
    /* istruzioni da ripetere */
}
```

La condizione viene controllata **prima** di ogni ripetizione. Se è subito falsa, il corpo non viene mai eseguito.

```c
int n;
printf("Inserisci un numero positivo: ");
scanf("%d", &n);

while (n <= 0) {
    printf("Il numero deve essere positivo. Riprova: ");
    scanf("%d", &n);
}

printf("Hai inserito: %d\n", n);
```

Questo ciclo continua a chiedere il numero finché l'utente non inserisce un valore valido. Non sai quante volte l'utente sbaglierà — per questo usi `while` e non `for`.

7.3.1 while per leggere finché l'utente vuole

```c
int valore;
int somma = 0;
int quanti = 0;

printf("Inserisci numeri (0 per finire):\n");
scanf("%d", &valore);

while (valore != 0) {
    somma += valore;
    quanti++;
    scanf("%d", &valore);
}

if (quanti > 0) {
    printf("Somma: %d\n", somma);
    printf("Media: %.2f\n", (double)somma / quanti);
} else {
    printf("Nessun numero inserito.\n");
}
```

! Il ciclo infinito

Se la condizione del `while` non diventa mai falsa, il programma non si ferma mai. Questo si chiama **ciclo infinito** ed è quasi sempre un errore.

```c
int i = 1;
while (i <= 10) {
    printf("%d\n", i);
    /* manca i++ - i resta sempre 1 - ciclo infinito! */
}
```

Assicurati sempre che qualcosa nel corpo del ciclo faccia progredire verso la condizione di uscita.
Per fermare un programma bloccato in un ciclo infinito: `Ctrl+C` nel terminale.

7.4 do-while — eseguire almeno una volta

Il ciclo `do-while` è simile al `while`, ma con una differenza importante: la condizione viene controllata **dopo** il corpo. Questo significa che il corpo viene eseguito **almeno una volta**, anche se la condizione è subito falsa.

```
do {
    /* istruzioni da ripetere */
} while (condizione);
```

È perfetto per i menu e per chiedere input all'utente almeno una volta:

```
int scelta;

do {
    printf("\n=== MENU ===\n");
    printf("1. Nuova partita\n");
    printf("2. Carica partita\n");
    printf("3. Esci\n");
    printf("Scelta: ");
    scanf("%d", &scelta);

    if (scelta == 1) {
        printf("Avvio nuova partita...\n");
    } else if (scelta == 2) {
        printf("Caricamento...\n");
    } else if (scelta != 3) {
        printf("Scelta non valida.\n");
    }

} while (scelta != 3);

printf("Arrivederci!\n");
```

Il menu viene mostrato almeno una volta. Se l'utente sceglie 3, il ciclo si ferma; altrimenti il menu viene ripresentato.

7.5 Confronto tra i tre cicli

	`for`	`while`	`do-while`
Quando usarlo	Numero di ripetizioni noto	Numero di ripetizioni ignoto	Corpo eseguito almeno una volta
Condizione	controllata prima	controllata prima	controllata dopo
Caso tipico	scorrere array, contare	validazione input, leggere fino a sentinella	menu, input utente

---

7.6 break e continue

Due parole chiave che modificano il comportamento del ciclo.

7.6.1 break — esci dal ciclo

`break` interrompe immediatamente il ciclo e salta al codice successivo.

```
int i;
for (i = 1; i <= 10; i++) {
    if (i == 5) {
        break;   /* esce quando i vale 5 */
    }
    printf("%d ", i);
}
/* stampa: 1 2 3 4 */
```

7.6.2 continue — salta alla prossima iterazione

`continue` salta il resto del corpo e passa alla prossima iterazione.

```
int i;
for (i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        continue;   /* salta i numeri pari */
    }
```

```
    printf("%d ", i);
}
/* stampa: 1 3 5 7 9 */
```

> **Usa break e continue con moderazione**
>
> `break` e `continue` sono utili ma rendono il flusso del programma più difficile da seguire se usati troppo. Prima di usarli, chiediti se puoi ottenere lo stesso risultato modificando la condizione del ciclo.

7.7 Cicli annidati

Puoi mettere un ciclo dentro un altro ciclo. Per ogni iterazione del ciclo esterno, il ciclo interno completa tutte le sue iterazioni.

```
int i, j;
for (i = 1; i <= 3; i++) {
    for (j = 1; j <= 3; j++) {
        printf("%d*%d=%d  ", i, j, i*j);
    }
    printf("\n");
}
```

Output:

```
1*1=1  1*2=2  1*3=3
2*1=2  2*2=4  2*3=6
3*1=3  3*2=6  3*3=9
```

I cicli annidati sono fondamentali per lavorare con le matrici (array bidimensionali) — lo vedremo nella lezione sugli array.

7.8 Esempio completo: indovina il numero

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int segreto, tentativo, tentativi;

    /* Genera un numero casuale tra 1 e 100 */
    srand(time(NULL));
    segreto = rand() % 100 + 1;

    tentativi = 0;

    printf("Ho scelto un numero tra 1 e 100. Indovina!\n\n");

    do {
        printf("Tentativo: ");
        scanf("%d", &tentativo);
        tentativi++;

        if (tentativo < segreto) {
            printf("Troppo basso!\n");
        } else if (tentativo > segreto) {
            printf("Troppo alto!\n");
        } else {
            printf("Esatto! Hai indovinato in %d tentativi.\n", tentativi);
        }

    } while (tentativo != segreto);

    return 0;
}
```

7.9 Schema mentale

So quante volte ripetere?

```
    → for (i = 0; i < n; i++)

Non so quante volte ripetere, ma so quando fermarmi?
    → while (condizione) { ... }
    → assicurati che qualcosa nel ciclo faccia progredire verso l'uscita

Devo eseguire il corpo almeno una volta (es. menu)?
    → do { ... } while (condizione);

Devo uscire dal ciclo prima della fine?
    → break

Devo saltare un'iterazione?
    → continue
```

7.10 Esercizi

Esercizio 1 Scrivi un programma che stampa la tavola pitagorica del numero inserito dall'utente.

```
Inserisci un numero: 7
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Esercizio 2 Scrivi un programma che legge numeri interi finché l'utente inserisce 0, poi stampa quanti numeri positivi e quanti negativi ha inserito.

Esercizio 3 Scrivi un programma che legge un numero intero `n` e stampa i primi `n` numeri di Fibonacci.

```
n=8 → 0 1 1 2 3 5 8 13
```

Esercizio 4 Scrivi un programma con un menu che permette di: 1. Calcolare la somma di N numeri 2. Trovare il massimo tra N numeri 3. Uscire

Il menu deve ripresentarsi finché l'utente non sceglie di uscire.

8 Funzioni

8.1 Perché le funzioni

Immagina di dover calcolare il massimo tra due numeri in dieci punti diversi del tuo programma. Potresti scrivere lo stesso `if` dieci volte — ma se ti accorgi di un errore, devi correggerlo in dieci posti. E se dimentichi uno?

Le **funzioni** risolvono questo problema: scrivi il codice una volta sola, gli dai un nome, e lo richiami ogni volta che ti serve.

Una funzione è un blocco di codice che: - ha un **nome** - riceve dei dati in ingresso (i **parametri**) - svolge un compito preciso - può restituire un risultato

---

8.2 Il ruolo del main

Prima di vedere come si scrivono le funzioni, è utile chiarire il ruolo del `main`.

Il `main` serve per tre cose: 1. leggere i dati 2. chiamare le funzioni 3. stampare i risultati

Le funzioni invece si occupano di **calcolare** e **trasformare**. Tenere separati questi ruoli rende il codice più chiaro e più facile da correggere.

> **Printf dentro una funzione**
>
> In generale è meglio evitare `printf` dentro una funzione. Una funzione dovrebbe fare un solo lavoro: calcolare o trasformare un dato. La stampa è responsabilità del `main`.

```c
/* Meglio così */
int massimo(int a, int b) {
    if (a > b) return a;
    return b;
}

int main() {
    int m = massimo(10, 7);
    printf("Il massimo è %d\n", m);   /* stampa nel main */
    return 0;
}
```

8.3 Struttura di una funzione

```c
tipo_ritorno nome_funzione(tipo parametro1, tipo parametro2, ...) {
    /* corpo della funzione */
    return valore;   /* se il tipo di ritorno non è void */
}
```

Tre decisioni da prendere ogni volta che scrivi una funzione:

1. Cosa deve restituire? - un numero (`int`, `double`) — usa `return valore` - niente (modifica una variabile, stampa qualcosa) — usa `void`

2. Cosa riceve in ingresso? - i parametri — i dati su cui lavora

3. Cosa fa? - il corpo — le istruzioni

8.4 Funzioni che restituiscono un valore

Usa `int` quando la funzione deve restituire un conteggio, una posizione, un valore numerico, oppure 1 o 0 (vero/falso).

```c
/* Restituisce il massimo tra due interi */
int massimo(int a, int b) {
    if (a > b) {
        return a;
    }
    return b;
}

/* Restituisce 1 se n è pari, 0 altrimenti */
int isPari(int n) {
    return n % 2 == 0;
}

/* Restituisce la media di due valori */
double media(double a, double b) {
    return (a + b) / 2.0;
}
```

Come si usano nel `main`:

```c
int main() {
    int x = 10, y = 7;
    printf("Massimo: %d\n", massimo(x, y));   /* 10 */

    if (isPari(x)) {
        printf("%d è pari\n", x);
    }

    printf("Media: %.2f\n", media(3.5, 7.5));  /* 5.50 */

    return 0;
}
```

8.5 Funzioni void

Usa `void` quando la funzione non restituisce un numero ma produce un risultato in altro modo — ad esempio modifica una stringa passata come parametro, o produce output.

```c
/* Stampa una riga di separazione */
void stampaSeparatore(int lunghezza) {
    int i;
    for (i = 0; i < lunghezza; i++) {
        printf("-");
    }
    printf("\n");
}

/* Stampa le statistiche (esempio di quando printf in funzione ha senso) */
void stampaStatistiche(int minimo, int massimo, double media) {
    printf("Minimo:  %d\n", minimo);
    printf("Massimo: %d\n", massimo);
    printf("Media:   %.2f\n", media);
}
```

Le funzioni `void` non hanno `return` (o hanno `return;` senza valore per uscire prima).

8.6 Il prototipo

In C il compilatore legge il codice dall'alto verso il basso. Se chiami una funzione prima di averla definita, il compilatore non la conosce ancora e dà errore.

La soluzione è il **prototipo**: una dichiarazione della funzione che va messa in cima al file, prima del `main`. Dice al compilatore "questa funzione esiste, la trovi più avanti".

```c
#include <stdio.h>

/* Prototipi - dichiarazioni anticipate */
int massimo(int a, int b);
int isPari(int n);
void stampaSeparatore(int lunghezza);

int main() {
    /* Qui puoi chiamare le funzioni anche se sono definite dopo */
    printf("Massimo: %d\n", massimo(10, 7));
    stampaSeparatore(20);
    return 0;
}
```

```c
/* Definizioni complete delle funzioni */
int massimo(int a, int b) {
    if (a > b) return a;
    return b;
}

int isPari(int n) {
    return n % 2 == 0;
}

void stampaSeparatore(int lunghezza) {
    int i;
    for (i = 0; i < lunghezza; i++) printf("-");
    printf("\n");
}
```

Struttura consigliata di un file C

```
1. #include delle librerie
2. #define delle costanti
3. Prototipi delle funzioni
4. main()
5. Definizioni delle funzioni
```

Seguire sempre questo ordine rende il codice prevedibile e facile da navigare.

8.7 Parametri: passaggio per valore

In C i parametri vengono passati **per valore**: la funzione riceve una **copia** del valore, non la variabile originale. Modificare un parametro dentro la funzione non cambia la variabile del chiamante.

```c
void raddoppia(int x) {
    x = x * 2;   /* modifica solo la copia locale */
}

int main() {
```

```
    int n = 5;
    raddoppia(n);
    printf("%d\n", n);   /* stampa 5 - n non è cambiato */
    return 0;
}
```

Se vuoi che la funzione modifichi la variabile originale, devi usare i **puntatori** — che vedremo nella prossima lezione.

8.8 Variabili locali e scope

Le variabili dichiarate dentro una funzione si chiamano **variabili locali**. Esistono solo dentro quella funzione — non sono visibili dall'esterno.

```
int somma(int a, int b) {
    int risultato = a + b;   /* risultato è locale a somma */
    return risultato;
}

int main() {
    int s = somma(3, 4);
    /* printf("%d\n", risultato);  ERRORE - risultato non esiste qui */
    printf("%d\n", s);   /* 7 */
    return 0;
}
```

Ogni funzione ha il proprio **scope** — il proprio spazio di variabili, separato da tutte le altre funzioni. Questo è uno dei vantaggi delle funzioni: non si intralciano a vicenda.

8.9 Strategia per scrivere una funzione

Quando vedi un esercizio che richiede una funzione, procedi così:

```
1. Capisci cosa deve produrre
   → un numero?      → int o double
   → niente?         → void

2. Capisci cosa riceve
   → quali dati le servono per lavorare?
   → quanti parametri e di che tipo?

3. Scrivi lo scheletro
   tipo nome(parametri) {
       ...
       return ...;
   }

4. Scrivi il corpo
   → il ciclo, la condizione, il calcolo

5. Scrivi il prototipo sopra il main

6. Chiamala nel main
```

8.10 Esempio completo

```c
#include <stdio.h>

/* Prototipi */
int massimo(int a, int b);
int minimo(int a, int b);
double media3(int a, int b, int c);
int isPari(int n);
void stampaRisultati(int a, int b, int c);

int main() {
    int a, b, c;

    printf("Inserisci tre numeri interi: ");
    scanf("%d %d %d", &a, &b, &c);
```

```c
    stampaRisultati(a, b, c);

    return 0;
}

int massimo(int a, int b) {
    return (a > b) ? a : b;
}

int minimo(int a, int b) {
    return (a < b) ? a : b;
}

double media3(int a, int b, int c) {
    return (a + b + c) / 3.0;
}

int isPari(int n) {
    return n % 2 == 0;
}

void stampaRisultati(int a, int b, int c) {
    int max = massimo(massimo(a, b), c);
    int min = minimo(minimo(a, b), c);
    double m = media3(a, b, c);

    printf("\n--- RISULTATI ---\n");
    printf("Massimo: %d\n", max);
    printf("Minimo:  %d\n", min);
    printf("Media:   %.2f\n", m);
    printf("Pari:    ");
    if (isPari(a)) printf("%d ", a);
    if (isPari(b)) printf("%d ", b);
    if (isPari(c)) printf("%d ", c);
    printf("\n");
}
```

8.11 Schema mentale

```
Cosa deve restituire la funzione?
    → un numero (conteggio, risultato, 1/0) → int o double + return
    → niente → void

Cosa riceve in ingresso?
    → una variabile → tipo nome
    → due variabili → tipo nome1, tipo nome2

Come la chiamo?
    → risultato = nomeFunzione(argomenti)   se restituisce qualcosa
    → nomeFunzione(argomenti)               se è void

Dove la metto nel file?
    → prototipo in cima
    → definizione dopo il main
    → chiamata nel main
```

8.12 Esercizi

Esercizio 1 Scrivi una funzione `int potenza(int base, int esponente)` che calcola `base` elevato a `esponente` usando un ciclo (non usare `pow` della libreria).

Esercizio 2 Scrivi una funzione `int isPrimo(int n)` che restituisce 1 se `n` è primo, 0 altrimenti. Chiamala nel `main` per stampare tutti i numeri primi da 1 a 100.

Esercizio 3 Scrivi le funzioni: - `int sommaDigiti(int n)` — restituisce la somma delle cifre di un numero - `int inverti(int n)` — restituisce il numero con le cifre invertite

Esempio: `sommaDigiti(347)` → `14`; `inverti(347)` → `743`

Esercizio 4 Scrivi un programma con un menu che chiama funzioni diverse in base alla scelta dell'utente: 1. Calcola il massimo tra due numeri 2. Verifica se un numero è primo 3. Calcola la somma delle cifre 4. Esci

9 Il tipo booleano

9.1 Vero e falso in C

In C non esiste un tipo booleano nativo come in altri linguaggi. Il C usa i numeri interi per rappresentare vero e falso:

- **0** significa **falso**
- **qualsiasi valore diverso da 0** significa **vero** (tipicamente si usa 1)

Questo è il motivo per cui le condizioni degli `if` e dei `while` sono espressioni numeriche:

```
if (5 > 3) { ... }     /* 5 > 3 vale 1 - vero */
if (0) { ... }          /* 0 - falso - il blocco non viene mai eseguito */
if (42) { ... }         /* 42 - vero - il blocco viene sempre eseguito */
```

E questo è il motivo per cui le funzioni che restituiscono vero/falso usano `int`:

```
int isPari(int n) {
    return n % 2 == 0;   /* restituisce 1 (vero) o 0 (falso) */
}
```

9.2 La libreria stdbool.h

Dal C99 in poi esiste la libreria `stdbool.h` che introduce il tipo `bool` e le costanti `true` e `false`. Rende il codice più leggibile senza cambiare nulla nel funzionamento.

```
#include <stdbool.h>
```

Con questa libreria puoi scrivere:

```
bool trovato = false;
bool finito = true;
```

invece di:

```
int trovato = 0;
int finito = 1;
```

9.3 bool, true e false

Con `stdbool.h`	Equivale a
`bool`	`int`
`true`	`1`
`false`	`0`

Sono solo nomi più leggibili — sotto al cofano sono ancora interi.

```
#include <stdio.h>
#include <stdbool.h>

bool isPari(int n) {
    return n % 2 == 0;
}

bool isPrimo(int n) {
    int i;
    if (n < 2) return false;
    for (i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

int main() {
    int n;
    printf("Inserisci un numero: ");
```

```
    scanf("%d", &n);

    if (isPari(n)) {
        printf("%d è pari.\n", n);
    } else {
        printf("%d è dispari.\n", n);
    }

    if (isPrimo(n)) {
        printf("%d è primo.\n", n);
    } else {
        printf("%d non è primo.\n", n);
    }

    return 0;
}
```

9.4 Usare bool come flag

Un uso molto comune di `bool` è il **flag** — una variabile che tiene traccia di una condizione che può cambiare durante il programma.

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int numeri[] = {3, 7, 12, 5, 8, 21, 4};
    int dimensione = 7;
    int cercato = 12;
    bool trovato = false;
    int posizione = -1;
    int i;

    for (i = 0; i < dimensione; i++) {
        if (numeri[i] == cercato) {
            trovato = true;
            posizione = i;
            break;
```

```
        }
    }

    if (trovato) {
        printf("%d trovato alla posizione %d.\n", cercato, posizione);
    } else {
        printf("%d non trovato.\n", cercato);
    }

    return 0;
}
```

Il flag `trovato` parte come `false`. Appena il numero viene trovato, diventa `true` e il ciclo si interrompe. Alla fine del ciclo, basta controllare il flag per sapere se la ricerca ha avuto successo.

9.5 bool nelle funzioni

Le funzioni che rispondono a una domanda sì/no si scrivono naturalmente con `bool`:

```
bool isVocale(char c) {
    c = tolower(c);   /* converte in minuscolo - da ctype.h */
    return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u';
}

bool isAlfa(char c) {
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z');
}

bool isUguali(int a[], int b[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        if (a[i] != b[i]) return false;
    }
    return true;
}
```

Convenzione sui nomi

Le funzioni che restituiscono `bool` di solito iniziano con `is` o `ha` — per comunicare chiaramente che rispondono a una domanda:

- `isPari`, `isPrimo`, `isVocale` — "è…?"
- `isAlfa`, `isDigit` — "è…?"
- `haSpazi`, `haVocali` — "ha…?"

Questa convenzione rende il codice molto più leggibile:

```
if (isPrimo(n)) { ... }   /* si legge quasi come italiano */
```

9.6 bool o int?

In pratica, molti programmi C usano ancora `int` per i valori booleani — soprattutto nel codice più vecchio. Entrambi gli approcci funzionano.

	`int`	`bool`
Disponibilità	sempre	richiede `#include <stdbool.h>`
Leggibilità	meno chiaro	più chiaro
Compatibilità	tutti i compilatori	C99 o successivo

Quale scegliere?

Usa `bool` quando vuoi rendere chiaro che una variabile o una funzione rappresenta un valore vero/falso. Usa `int` quando il risultato è usato anche come numero (conteggio, indice, ecc.).
In ogni caso, ricorda che sono la stessa cosa — `bool` è solo un nome più leggibile per `int`.

9.7 Schema mentale

```
La mia funzione risponde a una domanda sì/no?
```

```
    → usa bool come tipo di ritorno
    → chiamala con is... o ha...
    → return true / return false

Ho una variabile che tiene traccia di una condizione?
    → usala come flag: bool trovato = false
    → quando la condizione si verifica: trovato = true
    → alla fine controlla il flag con if (trovato)

Devo includere stdbool.h?
    → sì, se usi bool, true o false
    → no, se usi int, 1 e 0
```

9.8 Esercizio

Scrivi le seguenti funzioni usando `bool`:

- `bool isLettera(char c)` — true se `c` è una lettera dell'alfabeto
- `bool isMaiuscola(char c)` — true se `c` è una lettera maiuscola
- `bool isMinuscola(char c)` — true se `c` è una lettera minuscola
- `bool isCifra(char c)` — true se `c` è una cifra da '0' a '9'

Scrivi un `main` che legge un carattere e stampa tutte le informazioni su di esso:

```
Carattere: A
È una lettera?   sì
È maiuscola?     sì
È minuscola?     no
È una cifra?     no
```

Part II

Strutture dati

10 Array

10.1 Blocco 1: Dichiarare e accedere a un array

Un array è una sequenza di variabili dello stesso tipo, tutte con lo stesso nome, accessibili tramite un indice.

Senza array, per memorizzare 5 voti dovresti scrivere:

```
int voto1, voto2, voto3, voto4, voto5;
```

Con un array:

```
int voti[5];
```

Un solo nome, cinque celle. Se i voti fossero cento, cambieresti solo il numero.

10.1.1 Dichiarazione

```
int voti[5];           /* array di 5 interi */
double temperature[10]; /* array di 10 double */
char lettere[26];       /* array di 26 caratteri */
```

Il numero dentro le parentesi quadre è la **dimensione** — quante celle ha l'array. Deve essere un valore costante.

10.1.2 Accesso agli elementi

Ogni elemento ha un **indice** — un numero che indica la sua posizione. In C gli indici partono da **0**, non da 1.

```c
int voti[5];

voti[0] = 7;   /* primo elemento */
voti[1] = 8;   /* secondo elemento */
voti[2] = 6;   /* terzo elemento */
voti[3] = 9;   /* quarto elemento */
voti[4] = 7;   /* quinto elemento */

printf("%d\n", voti[0]);   /* stampa 7 */
printf("%d\n", voti[2]);   /* stampa 6 */
```

! Gli indici partono da 0

Un array di dimensione `N` ha indici da `0` a `N-1`. L'indice `N` non esiste — accedervi è un errore che può causare comportamenti imprevedibili.

```c
int voti[5];
voti[5] = 10;   /* ERRORE - l'indice 5 è fuori dall'array */
```

10.1.3 Inizializzazione

Puoi inizializzare un array al momento della dichiarazione:

```c
int voti[5] = {7, 8, 6, 9, 7};

/* Se dai meno valori, il resto viene inizializzato a 0 */
int voti[5] = {7, 8};   /* voti[2], voti[3], voti[4] valgono 0 */

/* Trucco per azzerare tutto l'array */
int voti[5] = {0};
```

10.1.4 Usare #define per la dimensione

È buona pratica definire la dimensione dell'array con una costante:

```c
#define N 5

int voti[N];
```

Così se devi cambiare la dimensione, la modifichi in un solo posto.

10.2 Blocco 2: Saper scorrere un array

Questo è il cuore di quasi tutto: se non sai fare questo, non puoi fare né somme né ricerche né ordinamenti.

Lo schema mentale deve essere: **accedo a un elemento alla volta, dall'indice 0 fino all'ultimo**.

```
#define N 5
int voti[N];
int i;

for (i = 0; i < N; i++) {
    /* lavoro su voti[i] */
}
```

Il ciclo `for` è perfetto per gli array perché sai sempre quanti elementi ci sono.

10.2.1 Leggere un array dall'utente

```
#define N 5
int voti[N];
int i;

for (i = 0; i < N; i++) {
    printf("Inserisci voto %d: ", i + 1);
    scanf("%d", &voti[i]);
}
```

10.2.2 Stampare un array

```
for (i = 0; i < N; i++) {
    printf("voti[%d] = %d\n", i, voti[i]);
}
```

10.2.3 Calcolare la somma

```
int somma = 0;
for (i = 0; i < N; i++) {
    somma += voti[i];
}
printf("Somma: %d\n", somma);
printf("Media: %.2f\n", (double)somma / N);
```

10.2.4 Trovare il massimo

```
int max = voti[0];   /* parti dal primo elemento */
for (i = 1; i < N; i++) {
    if (voti[i] > max) {
        max = voti[i];
    }
}
printf("Massimo: %d\n", max);
```

> **Parti sempre dal primo elemento**
>
> Quando cerchi il massimo o il minimo, inizializza la variabile con `voti[0]` — non con 0 o con un numero a caso. Poi scorri dal secondo elemento (`i = 1`) in poi.

10.3 Blocco 3: Come si scrive una funzione che lavora sugli array

Una funzione che lavora su un array riceve l'array e la sua dimensione come parametri.

```
tipo funzione(int v[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        /* lavoro su v[i] */
    }
}
```

Questo schema si adatta a qualsiasi esercizio sugli array.

10.3.1 Il tipo restituito

Si usa `int` quando la funzione deve restituire un conteggio, una posizione, un valore numerico, oppure 1 o 0 (trovato/non trovato).

```
int somma(int v[], int n);
int massimo(int v[], int n);
int cercaElemento(int v[], int n, int cercato);
```

Si usa `void` quando la funzione modifica l'array o non restituisce nulla.

```
void leggiArray(int v[], int n);
void stampaArray(int v[], int n);
void azzera(int v[], int n);
```

10.3.2 I parametri

Caso 1: array + dimensione

```
int somma(int v[], int n)
```

Serve per: analizzare, contare, trovare il massimo/minimo.

Caso 2: due array + dimensione

```
void copia(int src[], int dst[], int n)
```

Serve per: copiare, confrontare due array.

Caso 3: array + dimensione + valore

```
int cerca(int v[], int n, int elemento)
```

Serve per: cercare un elemento, contare le occorrenze di un valore.

10.3.3 Schema tipico di una funzione sugli array

```
tipo funzione(int v[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        /* lavoro su v[i] */
    }
    return ...;
}
```

Questo è il cuore dell'algoritmo.

10.3.4 Funzione che conta qualcosa

```
int contaPositivi(int v[], int n) {
    int i, cont = 0;
    for (i = 0; i < n; i++) {
        if (v[i] > 0)
            cont++;
    }
    return cont;
}
```

Esempi possibili: `contaNegativi`, `contaPari`, `contaMaggioriDi`.

10.3.5 Funzione che trova un valore

```
int massimo(int v[], int n) {
    int i, max = v[0];
    for (i = 1; i < n; i++) {
        if (v[i] > max)
            max = v[i];
    }
    return max;
}
```

10.3.6 Funzione che cerca un elemento

```c
int cerca(int v[], int n, int elemento) {
    int i;
    for (i = 0; i < n; i++) {
        if (v[i] == elemento)
            return i;   /* restituisce la posizione */
    }
    return -1;   /* non trovato */
}
```

> **Restituire -1 per "non trovato"**
>
> Quando una funzione cerca qualcosa e non lo trova, restituisce `-1`. È una convenzione universale in C — -1 non è mai un indice valido, quindi non crea ambiguità.

10.3.7 Funzione che modifica un array

```c
void azzera(int v[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        v[i] = 0;
    }
}
```

> **Gli array si passano per riferimento**
>
> A differenza dei tipi semplici, gli array non vengono copiati quando li passi a una funzione. La funzione lavora direttamente sull'array originale. Questo significa che le modifiche fatte dentro la funzione si riflettono sull'array del chiamante.
>
> ```c
> void raddoppia(int v[], int n) {
>  int i;
>  for (i = 0; i < n; i++) {
>  v[i] *= 2;   /* modifica l'array originale */
>  }
> }
> ```

10.4 Blocco 4: Array bidimensionali

Un array bidimensionale è una tabella — righe e colonne.

```
int matrice[3][4];   /* 3 righe, 4 colonne */
```

Si accede con due indici: prima la riga, poi la colonna.

```
matrice[0][0] = 1;   /* prima riga, prima colonna */
matrice[1][2] = 7;   /* seconda riga, terza colonna */
```

10.4.1 Scorrere una matrice con due cicli annidati

```
#define RIGHE 3
#define COLONNE 4

int matrice[RIGHE][COLONNE];
int i, j;

for (i = 0; i < RIGHE; i++) {
    for (j = 0; j < COLONNE; j++) {
        /* lavoro su matrice[i][j] */
    }
}
```

Lo schema mentale: **per ogni riga, scorri tutte le colonne**.

10.4.2 Leggere e stampare una matrice

```
/* Lettura */
for (i = 0; i < RIGHE; i++) {
    for (j = 0; j < COLONNE; j++) {
        printf("Elemento [%d][%d]: ", i, j);
        scanf("%d", &matrice[i][j]);
    }
}

/* Stampa */
```

```
for (i = 0; i < RIGHE; i++) {
    for (j = 0; j < COLONNE; j++) {
        printf("%4d", matrice[i][j]);
    }
    printf("\n");   /* vai a capo dopo ogni riga */
}
```

10.5 Blocco 5: Strategie per risolvere gli esercizi sugli array

Molti esercizi sugli array sembrano diversi, ma appartengono quasi sempre a quattro famiglie di problemi.

Quando vedi un esercizio, la prima cosa da fare è capire a quale famiglia appartiene.

Le quattro famiglie più comuni sono: **contare**, **trovare**, **cercare**, **trasformare**.

10.5.1 Esercizi di tipo contare

Bisogna scorrere l'array e contare quante volte succede qualcosa.

Esempi: contare i pari, contare i negativi, contare i valori maggiori di una soglia.

Strategia: - inizializzare un contatore - scorrere l'array - controllare se l'elemento soddisfa una condizione - aumentare il contatore - restituire il risultato

Schema mentale: **scorro l'array → se l'elemento soddisfa la condizione → aumento il contatore**

```
int contaPari(int v[], int n) {
    int i, cont = 0;
    for (i = 0; i < n; i++) {
        if (v[i] % 2 == 0)
            cont++;
    }
    return cont;
}
```

Varianti possibili: cambia solo la condizione. La struttura rimane la stessa.

10.5.2 Esercizi di tipo trovare

Bisogna trovare un valore particolare nell'array: il massimo, il minimo, la media, la somma.

Esempi: trovare il massimo, trovare il minimo, calcolare la media.

Strategia: - inizializzare la variabile risultato con il primo elemento - scorrere dal secondo elemento - aggiornare il risultato se l'elemento corrente è migliore - restituire il risultato

Schema mentale: **parto dal primo → confronto con il corrente → aggiorno se necessario**

```
int minimo(int v[], int n) {
    int i, min = v[0];
    for (i = 1; i < n; i++) {
        if (v[i] < min)
            min = v[i];
    }
    return min;
}
```

Varianti possibili: massimo, minimo, somma, media. La struttura rimane la stessa.

10.5.3 Esercizi di tipo cercare

Bisogna verificare se qualcosa è presente nell'array o trovare la sua posizione.

Esempi: verificare se un elemento è presente, trovare la prima occorrenza, verificare se l'array è ordinato.

Strategia: - scorrere l'array - confrontare ogni elemento con quello cercato - restituire la posizione se trovato, -1 altrimenti

Schema mentale: **scorro → confronto → se trovato restituisco la posizione → altrimenti -1**

```
int cerca(int v[], int n, int elemento) {
    int i;
    for (i = 0; i < n; i++) {
        if (v[i] == elemento)
            return i;
    }
    return -1;
}
```

Varianti possibili: trovare l'ultima occorrenza, contare le occorrenze, verificare se esiste. La logica è sempre confrontare elementi.

10.5.4 Esercizi di tipo trasformare

Bisogna modificare gli elementi dell'array o crearne uno nuovo trasformato.

Esempi: raddoppiare tutti gli elementi, azzerare i negativi, copiare solo i pari.

Strategia: - scorrere l'array - applicare la trasformazione a ogni elemento - scrivere il risultato nell'array originale o in uno nuovo

Schema mentale: **leggo un elemento → decido come trasformarlo → scrivo il risultato**

```
void raddoppia(int v[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        v[i] *= 2;
    }
}
```

Varianti possibili: azzerare i negativi, sostituire con il valore assoluto, copiare solo gli elementi che soddisfano una condizione.

Quando leggi una consegna, chiediti: - Devo contare qualcosa? → esercizio di conteggio - Devo trovare un valore (massimo, minimo, media)? → esercizio di ricerca del valore - Devo verificare se qualcosa è presente? → esercizio di ricerca - Devo modificare gli elementi? → esercizio di trasformazione

---

10.6 Esercizio — Contare

Scrivere la funzione `int contaPari(int v[], int n)` che restituisce il numero di elementi pari nell'array. Scrivere anche un main che: - legge N interi - chiama la funzione - stampa il risultato

Soluzione:

```c
#include <stdio.h>
#define N 5

int contaPari(int v[], int n) {
    int i, cont = 0;
    for (i = 0; i < n; i++) {
        if (v[i] % 2 == 0)
            cont++;
    }
    return cont;
}

int main() {
    int voti[N];
    int i;
    for (i = 0; i < N; i++) {
        printf("Inserisci elemento %d: ", i + 1);
        scanf("%d", &voti[i]);
    }
    printf("Elementi pari: %d\n", contaPari(voti, N));
    return 0;
}
```

10.7 Esercizio — Trovare

Scrivere la funzione `double media(int v[], int n)` che restituisce la media degli elementi dell'array. Scrivere anche un main che: - legge N interi - chiama la funzione - stampa il risultato con due decimali

Soluzione:

```c
#include <stdio.h>
#define N 5

double media(int v[], int n) {
    int i;
    double somma = 0;
    for (i = 0; i < n; i++) {
        somma += v[i];
    }
    return somma / n;
```

```
}

int main() {
    int voti[N];
    int i;
    for (i = 0; i < N; i++) {
        printf("Inserisci elemento %d: ", i + 1);
        scanf("%d", &voti[i]);
    }
    printf("Media: %.2f\n", media(voti, N));
    return 0;
}
```

10.8 Esercizio — Cercare

Scrivere la funzione `int cerca(int v[], int n, int elemento)` che restituisce la posizione dell'elemento cercato, oppure -1 se non è presente. Scrivere anche un main che: - legge N interi - legge il valore da cercare - chiama la funzione - stampa la posizione oppure "non trovato"

Soluzione:

```
#include <stdio.h>
#define N 5

int cerca(int v[], int n, int elemento) {
    int i;
    for (i = 0; i < n; i++) {
        if (v[i] == elemento)
            return i;
    }
    return -1;
}

int main() {
    int voti[N];
    int i, cercato, pos;
    for (i = 0; i < N; i++) {
        printf("Inserisci elemento %d: ", i + 1);
        scanf("%d", &voti[i]);
    }
    printf("Elemento da cercare: ");
```

```c
    scanf("%d", &cercato);
    pos = cerca(voti, N, cercato);
    if (pos != -1)
        printf("Trovato alla posizione %d\n", pos);
    else
        printf("Non trovato\n");
    return 0;
}
```

11 Matrici

11.1 Idea chiave

Una matrice in C è un **array di array**. Pensala come una tabella: righe e colonne. Ogni elemento è identificato da due indici: `[riga][colonna]`.

```
int m[3][4];
```

Significa: - 3 righe - 4 colonne - totale elementi = $3 \times 4 = 12$ - ogni elemento è `int`

È memoria contigua: come un array monodimensionale ma più grande. Gli elementi sono disposti riga per riga in memoria.

11.2 Blocco 1: Dichiarare e inizializzare una matrice

11.2.1 Dichiarazione

```
int m[3][4];          /* matrice di interi, 3 righe, 4 colonne */
double tabella[5][5]; /* matrice quadrata di double */
```

Con `#define` per le dimensioni — sempre consigliato:

```
#define RIGHE 3
#define COLONNE 4

int m[RIGHE][COLONNE];
```

11.2.2 Inizializzazione

Forma "tabella" — più leggibile:

```
int m[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

Forma lineare — equivalente:

```
int m[2][3] = {1, 2, 3, 4, 5, 6};
```

Entrambe generano la stessa struttura in memoria.

Azzerare tutto:

```
int m[3][4] = {0};   /* tutti gli elementi a 0 */
```

11.2.3 Accesso a un elemento

```
m[0][2]   /* riga 0, colonna 2 */
m[1][3]   /* riga 1, colonna 3 */
```

Prima l'indice riga, poi l'indice colonna. Come dire: "vai alla riga 0, poi alla colonna 2".

! Gli indici partono da 0

Una matrice `m[RIGHE][COLONNE]` ha righe da `0` a `RIGHE-1` e colonne da `0` a `COLONNE-1`. Uscire da questi limiti è un errore.

```
int m[3][4];
m[3][0] = 1;   /* ERRORE - la riga 3 non esiste */
m[0][4] = 1;   /* ERRORE - la colonna 4 non esiste */
```

11.3 Blocco 2: Saper scorrere una matrice

Questo è il cuore di quasi tutto: se non sai fare questo, non puoi fare né somme né ricerche.

Lo schema mentale deve essere: **per ogni riga, scorri tutte le colonne**.

```
int i, j;
for (i = 0; i < RIGHE; i++) {
    for (j = 0; j < COLONNE; j++) {
        /* lavoro su m[i][j] */
    }
}
```

11.3.1 Leggere una matrice dall'utente

```
int i, j;
for (i = 0; i < RIGHE; i++) {
    for (j = 0; j < COLONNE; j++) {
        printf("m[%d][%d]: ", i, j);
        scanf("%d", &m[i][j]);
    }
}
```

11.3.2 Stampare una matrice

```
int i, j;
for (i = 0; i < RIGHE; i++) {
    for (j = 0; j < COLONNE; j++) {
        printf("%4d", m[i][j]);
    }
    printf("\n");   /* a capo dopo ogni riga */
}
```

Output (per la matrice 3×3 con valori 1-9):

```
   1   2   3
   4   5   6
   7   8   9
```

11.3.3 Somma di tutti gli elementi

```
int somma = 0;
for (i = 0; i < RIGHE; i++) {
    for (j = 0; j < COLONNE; j++) {
        somma += m[i][j];
    }
}
printf("Somma: %d\n", somma);
```

11.3.4 Ricerca del massimo

```
int max = m[0][0];
for (i = 0; i < RIGHE; i++) {
    for (j = 0; j < COLONNE; j++) {
        if (m[i][j] > max)
            max = m[i][j];
    }
}
printf("Massimo: %d\n", max);
```

11.3.5 Somma di una riga specifica

```
int r = 1;   /* seconda riga */
int s = 0;
for (j = 0; j < COLONNE; j++) {
    s += m[r][j];
}
printf("Somma riga %d: %d\n", r, s);
```

11.3.6 Somma di una colonna specifica

```
int c = 2;   /* terza colonna */
int s = 0;
for (i = 0; i < RIGHE; i++) {
```

```
    s += m[i][c];
}
printf("Somma colonna %d: %d\n", c, s);
```

11.3.7 Diagonale principale (solo matrici quadrate)

```
/* La diagonale principale ha i == j */
int s = 0;
for (i = 0; i < RIGHE; i++) {
    s += m[i][i];
}
printf("Somma diagonale: %d\n", s);
```

11.4 Blocco 3: Funzioni con le matrici

Quando passi una matrice a una funzione, devi specificare almeno la dimensione delle colonne.

```
void stampa(int m[][4], int righe) {
    int i, j;
    for (i = 0; i < righe; i++) {
        for (j = 0; j < 4; j++) {
            printf("%4d", m[i][j]);
        }
        printf("\n");
    }
}
```

Uso: `stampa(m, 3);`

> **Perché bisogna specificare le colonne?**
>
> La memoria è contigua riga per riga. Per sapere dove inizia la riga `i`, il compilatore deve calcolare l'offset: `i * numero_colonne`. Senza sapere quante colonne ci sono, non può fare questo calcolo.
> Il numero di righe invece puoi passarlo come parametro, come facevi con gli array monodimensionali.

11.4.1 Schema delle funzioni più comuni

```
/* Somma tutti gli elementi */
int somma(int m[][COLONNE], int righe) {
    int i, j, s = 0;
    for (i = 0; i < righe; i++)
        for (j = 0; j < COLONNE; j++)
            s += m[i][j];
    return s;
}

/* Trova il massimo */
int massimo(int m[][COLONNE], int righe) {
    int i, j, max = m[0][0];
    for (i = 0; i < righe; i++)
        for (j = 0; j < COLONNE; j++)
            if (m[i][j] > max)
                max = m[i][j];
    return max;
}

/* Stampa la matrice */
void stampa(int m[][COLONNE], int righe) {
    int i, j;
    for (i = 0; i < righe; i++) {
        for (j = 0; j < COLONNE; j++)
            printf("%4d", m[i][j]);
        printf("\n");
    }
}
```

11.5 Blocco 4: sizeof e matrici

Puoi usare `sizeof` per calcolare dimensioni e indici — ma solo nello stesso scope in cui la matrice è dichiarata, non dentro le funzioni.

```c
int m[3][4];

int totale_byte  = sizeof(m);             /* 3*4*4 = 48 byte */
int byte_riga    = sizeof(m[0]);          /* 4*4   = 16 byte */
int byte_elem    = sizeof(m[0][0]);       /* 4 byte */

int righe    = sizeof(m)    / sizeof(m[0]);       /* 3 */
int colonne  = sizeof(m[0]) / sizeof(m[0][0]);    /* 4 */
```

sizeof non funziona dentro le funzioni

Quando passi una matrice a una funzione, il parametro decade a puntatore e `sizeof` non restituisce più la dimensione reale della matrice. Per questo si passa sempre il numero di righe come parametro separato.

```c
void funzione(int m[][4], int righe) {
    /* sizeof(m) qui NON dà la dimensione della matrice */
    /* usa il parametro righe e la costante 4 */
}
```

11.6 Esempio completo

```c
#include <stdio.h>

#define RIGHE   3
#define COLONNE 3

void leggi(int m[][COLONNE], int righe);
void stampa(int m[][COLONNE], int righe);
int  somma(int m[][COLONNE], int righe);
int  massimo(int m[][COLONNE], int righe);
void sommaDiagonale(int m[][COLONNE], int righe);

int main() {
    int m[RIGHE][COLONNE];

    printf("Inserisci la matrice %dx%d:\n", RIGHE, COLONNE);
```

```c
    leggi(m, RIGHE);

    printf("\nMatrice inserita:\n");
    stampa(m, RIGHE);

    printf("\nSomma totale:  %d\n", somma(m, RIGHE));
    printf("Massimo:       %d\n",  massimo(m, RIGHE));
    sommaDiagonale(m, RIGHE);

    return 0;
}

void leggi(int m[][COLONNE], int righe) {
    int i, j;
    for (i = 0; i < righe; i++)
        for (j = 0; j < COLONNE; j++) {
            printf("m[%d][%d]: ", i, j);
            scanf("%d", &m[i][j]);
        }
}

void stampa(int m[][COLONNE], int righe) {
    int i, j;
    for (i = 0; i < righe; i++) {
        for (j = 0; j < COLONNE; j++)
            printf("%4d", m[i][j]);
        printf("\n");
    }
}

int somma(int m[][COLONNE], int righe) {
    int i, j, s = 0;
    for (i = 0; i < righe; i++)
        for (j = 0; j < COLONNE; j++)
            s += m[i][j];
    return s;
}

int massimo(int m[][COLONNE], int righe) {
    int i, j, max = m[0][0];
    for (i = 0; i < righe; i++)
        for (j = 0; j < COLONNE; j++)
```

```
            if (m[i][j] > max)
                max = m[i][j];
    return max;
}

void sommaDiagonale(int m[][COLONNE], int righe) {
    int i, s = 0;
    for (i = 0; i < righe; i++)
        s += m[i][i];
    printf("Somma diagonale: %d\n", s);
}
```

11.7 Schema mentale

```
Devo lavorare su tutta la matrice?
    → due cicli annidati: for i (righe) → for j (colonne)

Devo lavorare su una sola riga r?
    → un ciclo: for j (colonne) → usa m[r][j]

Devo lavorare su una sola colonna c?
    → un ciclo: for i (righe) → usa m[i][c]

Devo lavorare sulla diagonale? (solo matrici quadrate)
    → un ciclo: for i → usa m[i][i]

Devo passare la matrice a una funzione?
    → specifica le colonne: int m[][COLONNE]
    → passa le righe come parametro
```

11.8 Esercizi

Esercizio 1 Scrivi un programma che legge una matrice 3×3 e stampa la somma di ogni riga e di ogni colonna.

Esercizio 2 Scrivi una funzione `int contieneValore(int m[][COLONNE], int righe, int val)` che restituisce 1 se il valore `val` è presente nella matrice, 0 altrimenti.

Esercizio 3 Scrivi una funzione `void trasposta(int m[][COLONNE], int t[][RIGHE], int righe)` che calcola la trasposta di una matrice (scambia righe e colonne).

Esercizio 4 Scrivi un programma che legge una matrice quadrata NxN e verifica se è simmetrica (cioè se `m[i][j] == m[j][i]` per ogni `i` e `j`).

12 Stringhe

12.1 Idea chiave

In C non esiste un tipo stringa. Una stringa è un **array di caratteri** che termina con il carattere speciale `'\0'` (chiamato terminatore nullo o null terminator).

```
char nome[10] = "Arthas";
```

In memoria si trova così:

```
'A' 'r' 't' 'h' 'a' 's' '\0' ? ? ?
 0    1    2    3    4    5    6
```

Il `'\0'` segnala la fine della stringa. Senza di esso il C non saprebbe dove finisce il testo.

! La dimensione deve contenere anche '\0'

Se la parola ha 6 caratteri, l'array deve avere almeno 7 celle — una in più per il terminatore.

```
char nome[7] = "Arthas";   /* 6 lettere + '\0' */
char nome[6] = "Arthas";   /* ERRORE - non c'è spazio per '\0' */
```

12.2 Perché non si usa scanf per leggere le stringhe

Prima di vedere come si legge una stringa, è importante capire perché `scanf` da solo non basta.

`scanf("%s", s)` legge caratteri fino al primo spazio o invio — poi si ferma. Se l'utente scrive `"ciao mondo"`, `scanf` legge solo `"ciao"` e lascia `" mondo"` nel buffer.

```
char s[100];
scanf("%s", s);
/* l'utente scrive: "ciao mondo" */
/* s contiene solo: "ciao" */
```

Per leggere una frase intera — spazi inclusi — si usa `fgets`.

12.3 Come funziona fgets

```
fgets(s, dimensione, stdin);
```

`fgets` riceve tre argomenti:

- `s` — l'array dove salvare la stringa
- `dimensione` — il numero massimo di caratteri da leggere, spazi inclusi
- `stdin` — da dove leggere (standard input = tastiera)

`fgets` legge caratteri dalla tastiera finché non incontra un invio (`\n`) oppure raggiunge il limite di `dimensione - 1` caratteri. Aggiunge sempre `'\0'` alla fine.

```
char s[10];
fgets(s, 10, stdin);
/* l'utente scrive: "ciao" + invio */
/* s contiene: "ciao\n\0" */
```

> **fgets salva anche il newline**
>
> `fgets` include il `\n` dell'invio nella stringa. Quindi se l'utente scrive `"ciao"` e preme invio, la stringa contiene `"ciao\n"` — non solo `"ciao"`.

In alcuni esercizi questo non dà fastidio. In altri bisogna eliminarlo. Lo vediamo nel Blocco 1.

12.3.1 fgets protegge dall'overflow

Il secondo argomento di `fgets` è fondamentale: dice quanti caratteri al massimo leggere. Se l'utente scrive più caratteri del previsto, `fgets` si ferma al limite — l'array non può andare in overflow.

```
char s[10];
fgets(s, 10, stdin);
/* anche se l'utente scrive 100 caratteri, fgets ne legge al massimo 9 + '\0' */
```

Con `scanf("%s", s)` invece non c'è nessun limite — se l'utente scrive troppo, si va fuori dall'array con conseguenze imprevedibili.

12.3.2 Usare sizeof per la dimensione

Invece di scrivere il numero a mano, usa `sizeof(s)` — così se cambi la dimensione dell'array non devi ricordare di aggiornare anche `fgets`:

```
char s[200];
fgets(s, sizeof(s), stdin);   /* preferibile a fgets(s, 200, stdin) */
```

12.3.3 Confronto scanf vs fgets

	`scanf("%s", s)`	`fgets(s, dim, stdin)`
Legge gli spazi	si ferma allo spazio	sì
Legge il newline	lo lascia nel buffer	lo include nella stringa
Protegge dall'overflow	no	sì, con il limite
Quando usarlo	una singola parola	una frase intera

Regola pratica

Quando devi leggere una stringa, usa sempre `fgets`. È più sicuro e legge anche le frasi con spazi.

```c
char s[200];
fgets(s, sizeof(s), stdin);
```

Se vuoi solo una parola senza spazi, `scanf("%s", s)` funziona — ma tieni presente che non ha protezione dall'overflow.

12.4 Blocco 1: Saper leggere correttamente una stringa

Devi saper fare senza esitazione questo schema: **leggo con `fgets`, se serve tolgo il newline**.

```c
char s[200];
fgets(s, sizeof(s), stdin);

/* se dobbiamo eliminare l'andata a capo */
s[strcspn(s, "\n")] = '\0';
```

Cose da sapere bene: - `fgets` legge anche gli spazi — a differenza di `scanf("%s", ...)` - salva anche `\n` alla fine — in alcuni casi va eliminato - la stringa finisce comunque con `'\0'`

12.4.1 Quando NON è necessario togliere `\n`

Non serve eliminarlo quando scorriamo la stringa carattere per carattere. Il `\n` verrà semplicemente trattato come un carattere qualsiasi.

```c
int i = 0;
while (s[i] != '\0') {
    /* lavoro su s[i] */
    i++;
}
```

Questo va bene negli esercizi in cui: - si contano caratteri (es: frequenza dei caratteri) - si analizza il testo (es: conteggio vocali) - si modificano solo alcune lettere (es: crittografia delle sole lettere) - si ignorano i caratteri non alfabetici

In questi casi il `\n` non crea problemi.

12.4.2 Quando invece bisogna togliere `\n`

È necessario eliminarlo quando vogliamo che la stringa rappresenti esattamente il testo inserito, senza l'invio finale. Questo succede quando:

confrontiamo stringhe

```
strcmp(s, "ciao")
```

Se la stringa è `"ciao\n"` il confronto fallisce.

calcoliamo la lunghezza "reale" Il newline verrebbe contato come carattere in più.

facciamo elaborazioni sensibili alla posizione Per esempio: palindromi, parsing preciso del testo, confronti esatti tra stringhe.

12.5 Blocco 2: Saper scorrere tutta la stringa

Questo è il cuore di quasi tutto: se non sai fare questo, non puoi fare né frequenze né crittografia.

```
int i = 0;
while (s[i] != '\0') {
    /* lavoro su s[i] */
    i++;
}
```

Lo schema mentale deve essere: **leggo un carattere alla volta finché trovo `'\0'`**.

12.6 Blocco 3: Saper riconoscere il tipo di carattere

Negli esercizi questo è fondamentale. Devi saper distinguere lettera maiuscola, lettera minuscola, carattere non alfabetico.

Schema minimo — senza librerie:

```c
if (s[i] >= 'A' && s[i] <= 'Z') {
    /* maiuscola */
} else if (s[i] >= 'a' && s[i] <= 'z') {
    /* minuscola */
} else {
    /* altro */
}
```

Con la libreria `ctype.h` — più leggibile:

```c
#include <ctype.h>

isupper(s[i])    /* 1 se maiuscola */
islower(s[i])    /* 1 se minuscola */
isalpha(s[i])    /* 1 se lettera */
isdigit(s[i])    /* 1 se cifra 0-9 */
isspace(s[i])    /* 1 se spazio, tab, \n */
toupper(s[i])    /* converte in maiuscolo */
tolower(s[i])    /* converte in minuscolo */
```

Trasformazione manuale — senza ctype.h:

```c
/* Da minuscolo a maiuscolo */
if (s[i] >= 'a' && s[i] <= 'z') {
    s[i] = s[i] - 'a' + 'A';   /* oppure: s[i] -= 32; */
}

/* Da maiuscolo a minuscolo */
if (s[i] >= 'A' && s[i] <= 'Z') {
    s[i] = s[i] - 'A' + 'a';   /* oppure: s[i] += 32; */
}
```

> **Perché funziona la trasformazione con i numeri?**
>
> I caratteri sono numeri — seguono la tabella ASCII. `'A'` vale 65, `'a'` vale 97. La differenza è sempre 32. Quindi `'a' - 'A' = 32` e `'A' + 32 = 'a'`. Per lo stesso motivo `'0'` vale 48, e `s[i] - '0'` converte un carattere cifra nel suo valore numerico.

12.7 Blocco 4: Le funzioni della libreria string.h

```
#include <string.h>
```

Le funzioni più usate:

Funzione	Cosa fa	Esempio
`strlen(s)`	lunghezza della stringa (senza `\0`)	`strlen("ciao")` → `4`
`strcpy(dst, src)`	copia `src` in `dst`	`strcpy(s2, s1)`
`strcat(dst, src)`	aggiunge `src` in fondo a `dst`	`strcat(s, " mondo")`
`strcmp(s1, s2)`	confronta due stringhe	`0` se uguali
`strcspn(s, rif)`	posizione del primo char di `rif` in `s`	trova `\n`

12.7.1 strcmp — confrontare due stringhe

`strcmp` restituisce: - `0` se le stringhe sono uguali - un valore negativo se `s1` viene prima di `s2` alfabeticamente - un valore positivo se `s1` viene dopo

```
if (strcmp(s1, s2) == 0) {
    printf("Stringhe uguali\n");
}
```

> ⚠ Non usare == per confrontare stringhe
>
> In C non puoi confrontare stringhe con `==` — confronteresti gli indirizzi in memoria, non il contenuto.
>
> ```
> if (s1 == s2) { ... }          /* SBAGLIATO */
> if (strcmp(s1, s2) == 0) { }   /* CORRETTO */
> ```

12.8 Blocco 5: Come si scrive una funzione che lavora sulle stringhe

Una funzione che lavora sulle stringhe svolge un compito preciso: contare qualcosa, trasformare una stringa, confrontare, cercare, convertire.

12.8.1 Le tre cose da decidere

1. Il tipo restituito

Si usa `int` quando la funzione deve restituire un conteggio, una posizione, un valore numerico, 1 oppure 0:

```
int contaMaiuscole(const char s[]);
int contaVocali(const char s[]);
int sottoStringa(const char s1[], const char s2[]);
```

Si usa `void` quando la funzione produce un risultato modificando una stringa:

```
void maiuscolo(const char s1[], char s2[]);
```

2. I parametri

Caso 1: una stringa

```
int contaMaiuscole(const char s[])
```

Serve per: contare, analizzare, convertire in place.

Caso 2: due stringhe

```
void maiuscolo(const char s1[], char s2[])
```

`s1` è la stringa di partenza, `s2` è la stringa risultato.

Caso 3: stringa + numero

```
void crittografa(const char s[], int k)
```

Serve per: traslare i caratteri, tagliare, cercare una posizione.

3. Lo schema del corpo

```
tipo funzione(const char s[]) {
    int i = 0;
    while (s[i] != '\0') {
        /* lavoro su s[i] */
        i++;
    }
}
```

Questo è il cuore dell'algoritmo.

12.8.2 Funzione che conta qualcosa

```
int funzione(const char s[]) {
    int i = 0, cont = 0;
    while (s[i] != '\0') {
        if (/* condizione */)
            cont++;
        i++;
    }
    return cont;
}
```

Esempi possibili: `contaMaiuscole`, `contaVocali`, `contaSpazi`.

12.8.3 Funzione che trasforma una stringa

```
void funzione(const char s1[], char s2[]) {
    int i = 0;
    while (s1[i] != '\0') {
        s2[i] = /* trasformazione di s1[i] */;
        i++;
    }
    s2[i] = '\0';   /* fondamentale - termina la nuova stringa */
}
```

12.8.4 Funzione che confronta due stringhe

```
int funzione(const char s1[], const char s2[]) {
    int i = 0;
    while (s1[i] != '\0' && s2[i] != '\0') {
        if (s1[i] != s2[i])
            return 0;
        i++;
    }
    return s1[i] == s2[i];   /* vero solo se entrambe finite insieme */
}
```

12.8.5 Il ruolo del main

Il main serve per: leggere i dati, chiamare le funzioni, stampare i risultati.

```
int main() {
    char s[100];
    printf("Inserisci una frase: ");
    fgets(s, 100, stdin);
    int risultato = contaMaiuscole(s);
    printf("Maiuscole: %d\n", risultato);
    return 0;
}
```

In generale è meglio evitare `printf` dentro una funzione. Una funzione dovrebbe fare un solo lavoro: calcolare o trasformare. La stampa è responsabilità del main. Separare queste due cose rende il programma più chiaro, più riutilizzabile, più facile da testare.

12.9 Blocco 6: Strategie per risolvere gli esercizi sulle stringhe

Molti esercizi sulle stringhe sembrano diversi, ma appartengono quasi sempre a quattro famiglie di problemi. Quando vedi un esercizio, la prima cosa da fare è capire a quale famiglia appartiene.

Le quattro famiglie sono: **contare**, **trasformare**, **cercare**, **convertire**.

12.9.1 Esercizi di tipo contare

Bisogna scorrere la stringa e contare quante volte succede qualcosa.

Esempi: contare le maiuscole, contare le vocali, contare gli spazi, contare la frequenza delle lettere, contare i caratteri speciali.

Strategia: - inizializzare un contatore - scorrere la stringa carattere per carattere - controllare se il carattere soddisfa una condizione - aumentare il contatore - restituire il risultato

Schema mentale: **scorro la stringa → se il carattere soddisfa la condizione → aumento il contatore**

Varianti possibili: la consegna può cambiare solo la condizione — se è una vocale, se è uno spazio, se è un numero, se è un carattere alfabetico. La struttura dell'algoritmo rimane la stessa.

12.9.2 Esercizi di tipo trasformare

Bisogna modificare i caratteri della stringa oppure creare una nuova stringa trasformata.

Esempi: convertire in maiuscolo, convertire in minuscolo, cifrare una stringa, sostituire un carattere, eliminare certi caratteri.

Strategia: - scorrere la stringa - controllare il carattere - se serve trasformarlo - scrivere il risultato nella nuova stringa (o nella stessa)

Schema mentale: **leggo un carattere → decido se modificarlo → scrivo il risultato**

Varianti possibili: sostituire le vocali con *, cifrare con uno spostamento, eliminare gli spazi, trasformare minuscole in maiuscole. La struttura resta sempre la stessa.

12.9.3 Esercizi di tipo cercare

Bisogna verificare se qualcosa è presente nella stringa.

Esempi: verificare se una parola è contenuta in una frase, trovare la posizione di un carattere, controllare se due stringhe sono uguali, verificare se una stringa è palindroma.

Strategia: - scorrere la stringa principale - confrontare i caratteri - verificare se la sequenza cercata appare - restituire 1 oppure 0

Schema mentale: **parto da una posizione → confronto i caratteri → se coincidono fino alla fine → trovato**

Varianti possibili: verificare se una parola è presente, trovare la prima posizione di una lettera, verificare se due stringhe sono uguali. La logica è sempre confrontare caratteri.

12.9.4 Esercizi di tipo convertire

Bisogna trasformare una stringa in un numero o in un altro formato.

Esempi: convertire un numero romano in intero, convertire una stringa numerica in un numero, interpretare un codice.

Strategia: - scorrere la stringa - riconoscere il simbolo corrente - assegnare il valore numerico - aggiornare il risultato finale

Schema mentale: **leggo simbolo → lo trasformo in numero → aggiorno il totale**

Quando leggi una consegna, chiediti: - Devo contare qualcosa? → esercizio di conteggio - Devo modificare la stringa? → esercizio di trasformazione - Devo verificare se qualcosa è presente? → esercizio di ricerca - Devo trasformare il testo in un numero? → esercizio di conversione

---

12.10 Esercizio — Contare

Scrivere la funzione `int contaSpazi(const char s[])` che restituisce il numero di spazi presenti nella stringa `s`. Scrivere anche un main che legge una frase, chiama la funzione e stampa il risultato.

Soluzione:

```
#include <stdio.h>

int contaSpazi(const char s[]) {
    int i = 0, cont = 0;
    while (s[i] != '\0') {
        if (s[i] == ' ')
            cont++;
        i++;
    }
    return cont;
}

int main() {
    char frase[200];
    printf("Inserisci una frase: ");
    fgets(frase, sizeof(frase), stdin);
    printf("Numero di spazi: %d\n", contaSpazi(frase));
    return 0;
}
```

12.11 Esercizio — Trasformare

Scrivere la funzione `void sostituisciVocali(const char s1[], char s2[])` che copia la stringa `s1` in `s2`, sostituendo tutte le vocali con `*`. Scrivere anche un main che legge una parola, chiama la funzione e stampa la stringa trasformata.

Soluzione:

```c
#include <stdio.h>

void sostituisciVocali(const char s1[], char s2[]) {
    int i = 0;
    while (s1[i] != '\0') {
        char c = s1[i];
        if (c=='a'||c=='e'||c=='i'||c=='o'||c=='u'||
            c=='A'||c=='E'||c=='I'||c=='O'||c=='U')
            s2[i] = '*';
        else
            s2[i] = c;
        i++;
    }
    s2[i] = '\0';
}

int main() {
    char parola[100], risultato[100];
    printf("Inserisci una parola: ");
    fgets(parola, sizeof(parola), stdin);
    sostituisciVocali(parola, risultato);
    printf("Risultato: %s\n", risultato);
    return 0;
}
```

12.12 Esercizio — Cercare

Scrivere la funzione `int contieneCarattere(const char s[], char c)` che restituisce 1 se il carattere `c` è presente nella stringa `s`, 0 altrimenti. Scrivere anche un main che legge una parola e un carattere, chiama la funzione e stampa il risultato.

Soluzione:

```c
#include <stdio.h>

int contieneCarattere(const char s[], char c) {
    int i = 0;
    while (s[i] != '\0') {
        if (s[i] == c)
            return 1;
        i++;
```

```
    }
    return 0;
}

int main() {
    char parola[100];
    char c;
    printf("Inserisci una parola: ");
    fgets(parola, sizeof(parola), stdin);
    printf("Inserisci un carattere: ");
    scanf(" %c", &c);
    printf("Risultato: %d\n", contieneCarattere(parola, c));
    return 0;
}
```

12.13 Esercizio — Convertire

Scrivere la funzione `int sommaCifre(const char s[])` che riceve una stringa contenente solo cifre numeriche e restituisce la somma delle cifre. Scrivere anche un main che legge la stringa, chiama la funzione e stampa il risultato.

Soluzione:

```
#include <stdio.h>

int sommaCifre(const char s[]) {
    int i = 0, somma = 0;
    while (s[i] != '\0') {
        somma += s[i] - '0';
        i++;
    }
    return somma;
}

int main() {
    char numero[100];
    printf("Inserisci un numero: ");
    fgets(numero, sizeof(numero), stdin);
    printf("Somma cifre: %d\n", sommaCifre(numero));
    return 0;
}
```

Noterai che tutti e quattro gli esercizi hanno lo stesso cuore

```
int i = 0;
while (s[i] != '\0') {
    /* lavoro su s[i] */
    i++;
}
```

Cambia solo cosa fai dentro il ciclo.

Part III

Puntatori

13 Puntatori

13.1 Idea chiave

Un puntatore è una variabile che contiene un **indirizzo di memoria**.

Non contiene direttamente un valore come `5` o `'A'`, ma dice **dove si trova** quel valore.

```
int x = 10;
int *p = &x;
```

Qui: - `x` è una variabile normale che contiene `10` - `&x` significa: indirizzo di `x` - `p` è un puntatore a intero - `p` contiene l'indirizzo di `x`

13.2 Blocco 1: Capire cos'è un puntatore

Devi distinguere bene tre cose:

```
la variabile       →    x
il suo indirizzo   →    &x
il valore puntato  →    *p
```

Riprendiamo l'esempio:

```
int x = 10;
int *p = &x;
```

Espressione	Significato	Valore
`x`	la variabile	`10`
`&x`	indirizzo di `x`	es. `0x7fff...`
`p`	contiene `&x`	es. `0x7fff...`
`*p`	valore all'indirizzo puntato	`10`

Cose da sapere bene: - `*` nella **dichiarazione** significa: "questa variabile è un puntatore" - `&` significa: "prendi l'indirizzo di questa variabile" - `*p` nell'**uso** significa: "vai all'indirizzo contenuto in `p` e leggi/modifica il valore"

> **Idea chiave**
>
> Se `p` punta a `x`, allora `*p` è un altro modo per accedere a `x`. Modificare `*p` modifica direttamente `x`.

13.3 Blocco 2: Saper leggere questa scrittura

Questa è la scrittura che devi saper leggere senza esitazione:

```
int x = 10;
int *p = &x;
```

Si legge così: **`p` è un puntatore a `int` e contiene l'indirizzo di `x`.**

Se poi scrivi:

```
printf("%d\n", *p);
```

significa: stampa il valore contenuto nella variabile puntata da `p` — quindi stampa `10`.

13.3.1 Modificare una variabile tramite puntatore

```
int x = 10;
int *p = &x;
*p = 25;
printf("%d\n", x);   /* stampa 25 */
```

Il risultato è 25 perché modificando `*p` stai modificando direttamente `x`.

13.4 Blocco 3: Operatori fondamentali

Con i puntatori ci sono due operatori essenziali.

13.4.1 Operatore `&` — prendi l'indirizzo

Serve per ottenere l'indirizzo di una variabile.

```
int x = 7;
printf("%p\n", &x);   /* stampa l'indirizzo di x */
```

13.4.2 Operatore `*` — due significati diversi

1. Nella dichiarazione:

```
int *p;   /* p è un puntatore a intero */
```

2. Nell'uso:

```
*p   /* valore contenuto nella cella di memoria puntata da p */
```

13.4.3 Esempio completo

```
int x = 7;
int *p = &x;

printf("%d\n", x);    /* 7          - valore di x */
printf("%p\n", &x);   /* indirizzo  - indirizzo di x */
printf("%p\n", p);    /* indirizzo  - p contiene lo stesso indirizzo */
printf("%d\n", *p);   /* 7          - valore puntato da p */
```

Cosa devi osservare: `&x` e `p` sono uguali come indirizzo. `x` e `*p` sono uguali come valore.

13.5 Blocco 4: Errori da non fare

13.5.1 Errore 1: usare un puntatore non inizializzato

```
int *p;
*p = 5;   /* SBAGLIATO */
```

Questo è sbagliato perché `p` non punta a nessuna zona valida di memoria. Stai dicendo: "vai all'indirizzo contenuto in `p` e scrivi `5`" — ma quell'indirizzo potrebbe essere una zona di memoria qualsiasi, memoria di un altro programma, una zona non accessibile.

Questo comportamento si chiama **undefined behavior** — il linguaggio C non garantisce cosa succede. Ci sono tre scenari possibili:

Caso 1: fortuna `p` contiene un indirizzo che esiste ed è scrivibile → il programma non crasha → sembra funzionare.

Caso 2: crash immediato Segmentation fault / access violation.

Caso 3: bug silenzioso *(il peggiore)* Scrivi in memoria "valida" ma sbagliata → il programma continua, ma è corrotto.

> ❗ Inizializza sempre i puntatori
>
> Prima di usare un puntatore, fallo puntare a una variabile valida con `&`, oppure impostalo a `NULL` per indicare che non punta a niente.
>
> ```
> int *p = NULL;    /* puntatore nullo - non punta a niente */
> int x = 10;
> int *q = &x;     /* puntatore valido - punta a x */
> ```

13.5.2 Errore 2: confondere indirizzo e valore

```
int x = 10;
int *p = &x;
printf("%d\n", p);    /* concettualmente sbagliato */
printf("%d\n", *p);   /* corretto per il valore */
```

Per stampare un indirizzo si usa `%p`:

```
printf("%p\n", p);    /* corretto per l'indirizzo */
```

13.5.3 Errore 3: tipo non compatibile

```
int x = 10;
char *p = &x;   /* SBAGLIATO - p è puntatore a char, &x è indirizzo di int */
```

Regola fondamentale: **un puntatore deve puntare a una variabile del tipo corretto**.

13.6 Blocco 5: Il legame tra puntatori e funzioni

Questo è il motivo per cui i puntatori sono importanti.

In C, una funzione riceve normalmente una **copia** dei parametri. Quindi se scrivo:

```
void modifica(int x) {
    x = 20;
}

int a = 10;
modifica(a);
printf("%d\n", a);   /* stampa 10 - a non è cambiato */
```

La funzione ha modificato solo la copia locale di `x` — `a` è rimasto invariato.

13.6.1 Come modificare davvero una variabile

Si passa il suo **indirizzo**.

```
void modifica(int *p) {
    *p = 20;
}

int a = 10;
modifica(&a);
printf("%d\n", a);   /* stampa 20 - a è cambiato */
```

Schema mentale:

```
se passi a    →  la funzione riceve il valore - non può modificare a
se passi &a   →  la funzione riceve l'indirizzo
se usa *p     →  modifica direttamente a
```

13.7 Blocco 6: Schema tipico di funzione con puntatore

13.7.1 Caso 1: modificare una variabile

```c
void raddoppia(int *p) {
    *p = (*p) * 2;
}

int x = 5;
raddoppia(&x);
printf("%d\n", x);   /* 10 */
```

13.7.2 Caso 2: scambiare due variabili

Questo è uno dei classici più importanti.

```c
void scambia(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int x = 3, y = 8;
scambia(&x, &y);
/* x vale 8, y vale 3 */
```

Perché servono i puntatori? Perché senza di essi la funzione scambierebbe solo le copie, non le variabili originali.

13.8 Blocco 7: Puntatori e array

Il nome di un array è strettamente collegato a un puntatore.

```
int v[5] = {10, 20, 30, 40, 50};
```

`v` rappresenta l'indirizzo del primo elemento dell'array — cioè `v` è l'indirizzo di `v[0]`.

13.8.1 Equivalenze fondamentali

```
v[0] == *v
v[1] == *(v + 1)
v[2] == *(v + 2)
```

```
int v[3] = {4, 7, 9};

printf("%d\n", v[0]);      /* 4 */
printf("%d\n", *v);        /* 4 - stesso valore */

printf("%d\n", v[1]);      /* 7 */
printf("%d\n", *(v + 1));  /* 7 - stesso valore */
```

13.8.2 Scorrere un array con un puntatore

```
int v[5] = {1, 2, 3, 4, 5};
int *p = v;   /* p punta al primo elemento */

for (i = 0; i < 5; i++) {
    printf("%d\n", *p);
    p++;   /* sposta il puntatore al prossimo elemento */
}
```

13.9 Schema mentale

```
Cosa contiene p?
    → un indirizzo di memoria

Come faccio puntare p a x?
    → p = &x

Come leggo il valore puntato?
    → *p

Come modifico il valore puntato?
    → *p = nuovo_valore

Voglio che una funzione modifichi una variabile?
    → passa &a, ricevi int *p, usa *p dentro la funzione

Puntatore e array?
    → v e &v[0] sono la stessa cosa
    → v[i] e *(v+i) sono equivalenti
```

13.10 Esercizi

13.10.1 Esercizio 1: Primo contatto

Dichiara una variabile `int x` e un puntatore `p`. Assegna a `x` un valore a tua scelta, fai puntare `p` a `x` e stampa: il valore di `x`, l'indirizzo di `x`, il valore di `p`, il valore puntato da `p`.

Soluzione:

```c
#include <stdio.h>

int main() {
    int x = 10;
    int *p = &x;

    printf("x = %d\n", x);
    printf("indirizzo di x = %p\n", &x);
    printf("p = %p\n", p);
```

```
    printf("valore puntato da p = %d\n", *p);

    return 0;
}
```

13.10.2 Esercizio 2: Modifica tramite puntatore

Dichiara `int x = 10`, usa un puntatore per modificarla a `25`, stampa il valore finale.

Soluzione:

```
#include <stdio.h>

int main() {
    int x = 10;
    int *p = &x;
    *p = 25;
    printf("%d\n", x);
    return 0;
}
```

13.10.3 Esercizio 3: Funzione che azzera

Scrivi `void azzera(int *p)` che imposta a 0 il valore della variabile.

Soluzione:

```
#include <stdio.h>

void azzera(int *p) {
    *p = 0;
}

int main() {
    int x = 7;
    azzera(&x);
    printf("%d\n", x);
    return 0;
}
```

13.10.4 Esercizio 4: Raddoppio

Scrivi `void raddoppia(int *p)` che raddoppia il valore.

Soluzione:

```
#include <stdio.h>

void raddoppia(int *p) {
    *p = (*p) * 2;
}

int main() {
    int x = 5;
    raddoppia(&x);
    printf("%d\n", x);
    return 0;
}
```

13.10.5 Esercizio 5: Scambio di due variabili

Scrivi `void scambia(int *a, int *b)`.

Soluzione:

```
#include <stdio.h>

void scambia(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 3, y = 8;
    scambia(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```

13.10.6 Esercizio 6: Puntatore e array

Dato `int v[5] = {10, 20, 30, 40, 50}`, stampa il primo elemento usando sia `v[0]` sia un puntatore `*p`.

Soluzione:

```
#include <stdio.h>

int main() {
    int v[5] = {10, 20, 30, 40, 50};
    int *p = v;

    printf("%d\n", v[0]);
    printf("%d\n", *p);

    return 0;
}
```

13.10.7 Esercizio 7: Scorrere un array con puntatore

Stampa tutti gli elementi usando un puntatore.

Soluzione:

```
#include <stdio.h>

int main() {
    int v[5] = {1, 2, 3, 4, 5};
    int *p = v;
    int i;

    for (i = 0; i < 5; i++) {
        printf("%d\n", *p);
        p++;
    }

    return 0;
}
```

13.10.8 Esercizio 8: Somma con puntatore

Scrivi `int somma(int v[], int n)` usando un puntatore internamente.

Soluzione:

```
#include <stdio.h>

int somma(int v[], int n) {
    int *p = v;
    int s = 0, i;

    for (i = 0; i < n; i++) {
        s += *p;
        p++;
    }

    return s;
}

int main() {
    int v[4] = {2, 4, 6, 8};
    printf("%d\n", somma(v, 4));
    return 0;
}
```

13.10.9 Esercizio 9: Indice vs puntatore

Scrivi due versioni per stampare l'array: una con `v[i]`, una con `*(v + i)`.

Soluzione:

```
#include <stdio.h>

int main() {
    int v[4] = {2, 4, 6, 8};
    int i;

    for (i = 0; i < 4; i++)
        printf("%d\n", v[i]);

    for (i = 0; i < 4; i++)
        printf("%d\n", *(v + i));
```

```
    return 0;
}
```

13.10.10 Esercizio 10: Massimo con puntatore

Scrivi `int massimo(int v[], int n)` usando un puntatore.

Soluzione:

```
#include <stdio.h>

int massimo(int v[], int n) {
    int *p = v;
    int max = *p;
    int i;

    for (i = 0; i < n; i++) {
        if (*p > max)
            max = *p;
        p++;
    }

    return max;
}

int main() {
    int v[5] = {3, 7, 2, 9, 5};
    printf("%d\n", massimo(v, 5));
    return 0;
}
```

13.10.11 Esercizio 11: Restituire un puntatore

Scrivi `int* trovaPari(int v[], int n)` che restituisce il puntatore al primo elemento pari, `NULL` se non esiste.

Soluzione:

```c
#include <stdio.h>

int* trovaPari(int v[], int n) {
    int *p = v;
    int i;

    for (i = 0; i < n; i++) {
        if (*p % 2 == 0)
            return p;
        p++;
    }

    return NULL;
}

int main() {
    int v[5] = {1, 3, 5, 8, 9};
    int *ris = trovaPari(v, 5);

    if (ris != NULL)
        printf("%d\n", *ris);
    else
        printf("Nessun pari\n");

    return 0;
}
```